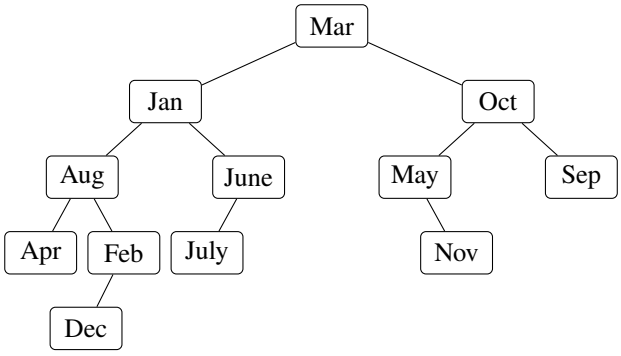
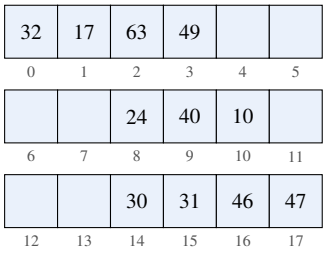
查找：二叉排序树、平衡二叉树、B 树与散列表

《数据结构与算法》第 8 章作业　学习讲义





内容结构　全部原题 → 全部答案与详解 → 专题拓展 → 参考资料

涵盖知识　平均查找长度 ASL ·二叉排序树·折半查找判定树·AVL 旋转 3 阶 B 树插入/分裂/删除/借位·散列函数·线性探测·聚集

编写日期　2026 年 11 月

目录

1 第一部分　全部原始作业题目 **2**

8.1 2

8.2 2

8.3 2

2 第二部分　全部答案与超详细解析 **3**

【第 8.1 题】 3

【第 8.2 题】 8

【第 8.3 题】 10

3 第三部分　专题拓展 **12**

开篇：题目分类与筛选 12

3.1 第一章　本次作业知识体系总览 12

3.2 第二章　重点题深度回顾 13

3.3 第三章　重点题举一反三 13

3.4 第四章　易错题专题 16

3.5 第五章　易错题举一反三 17

3.6 第六章　核心知识系统回顾 17

3.7 第七章　学科科普：我们学的“查找”究竟是什么 20

3.8 第八章　科学史：从“对半猜”到 B 树 20

3.9 第九章　关联学科知识网络与跨课程迁移 21

3.10 第十章　国内外高校公开试题典型题专题 22

3.11 第十一章　生活中的实际应用 26

3.12 第十二章　学习总结 28

4 第四部分　参考资料与权威来源 **29**

1 第一部分 全部原始作业题目

8.1

已知如下所示长度为 12 的表：

$$(\text{Jan},\ \text{Feb},\ \text{Mar},\ \text{Apr},\ \text{May},\ \text{June},\ \text{July},\ \text{Aug},\ \text{Sep},\ \text{Oct},\ \text{Nov},\ \text{Dec})$$

① 试按表中元素的顺序依次插入一棵初始为空的二叉排序树，画出插入完成之后的二叉排序树，并求其在等概率的情况下查找成功的平均查找长度。
② 若对表中元素先进行排序构成有序表，求在等概率的情况下对此有序表进行折半查找时查找成功的平均查找长度。
③ 按表中元素顺序构造一棵平衡二叉排序树，并求其在等概率的情况下查找成功的平均查找长度。

8.2

对下面的 3 阶 B-树（见图 1），依次执行下列操作，画出各步操作的结果。

① 插入90　② 插入25　③ 插入45　④ 删除60

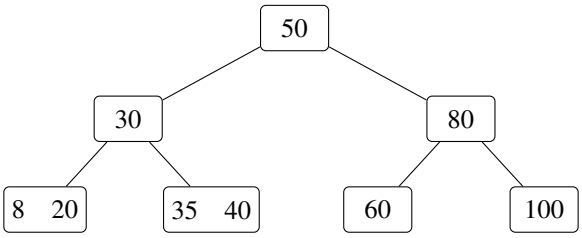


图 1: 8.2 题给出的 3 阶 B-树（按原题图重绘）

8.3

设哈希表的地址范围为 $0 \sim 17$，哈希函数为 $H(key) = key \ \% \ 16$。用线性探测法处理冲突，输入关键字序列

$$(10,\ 24,\ 32,\ 17,\ 31,\ 30,\ 46,\ 47,\ 40,\ 63,\ 49),$$

构造哈希表，试回答下列问题：

① 画出哈希表的示意图；
② 若查找关键字 63，需要依次与哪些关键字进行比较?
③ 若查找关键字 60，需要依次与哪些关键字比较?
④ 假定每个关键字的查找概率相等，求查找成功时的平均查找长度。

2 第二部分 全部答案与超详细解析

贯穿全章的统一约定

- **字符串比较**：月份名按字典序（逐字符比较 ASCII 码，大写字母 J < M 等；一个串是另一个串的前缀时，短串较小）。例如 June 与 July 前两个字符相同，第三个字符 `n` > `l`，故 July < June。
- **平均查找长度**（Average Search Length）：查找成功时 $\mathrm{ASL}=\sum_{i=1}^{n} p_i c_i$，其中 p_i 是查找第 i 个记录的概率，c_i 是找到它所需的关键字比较次数。等概率时 $p_i=1/n$，于是 $\mathrm{ASL}=\frac{1}{n}\sum c_i$。它本质上就是概率论中“比较次数这个随机变量的数学期望”。
- **树中层次**：根为第 1 层。在二叉排序树、平衡二叉树、折半查找判定树中，查找位于第 ℓ 层的结点恰好比较 ℓ 次，因此 $\mathrm{ASL}=\frac{1}{n}\sum$(各结点层次)。

【第 8.1 题】

① 二叉排序树及其 ASL

考查知识点 二叉排序树（BST）的定义与插入算法；字符串字典序；等概率查找成功的 ASL。

已知条件 12 个关键字的插入顺序固定：Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec；初始为空树；比较规则为字典序。

解题目标 画出最终 BST；求 $\mathrm{ASL}_{\text{成功}}$。

解题思路 BST 的形状完全由“插入顺序”决定：每个新关键字从根出发，比根小往左、比根大往右，走到空位置就挂上去。所以只要把 12 个关键字按顺序逐个“走一遍”，记下它停在第几层；最后把层次相加除以 12 即为 ASL。

定义：二叉排序树（二叉查找树，BST）

空树，或满足：(1) 左子树非空时，左子树上所有结点的关键字均小于根；(2) 右子树非空时，右子树上所有结点的关键字均大于根；(3) 左、右子树也分别是 BST。
插入算法：若树空，新结点作根；否则与根比较，小则递归插入左子树，大则递归插入右子树。新结点总是作为**叶子**插入，不移动已有结点。
为什么可用：本题关键字互不相同且有全序（字典序），满足 BST 的使用条件。

完整推导 逐个插入（“<”“>”均为字典序比较）：

序	关键字	比较路径	所在层
1	Jan	空树，作根	1
2	Feb	Feb < Jan（F<J）→ Jan 的左孩子	2
3	Mar	Mar > Jan → Jan 的右孩子	2
4	Apr	Apr < Jan → Apr < Feb → Feb 的左孩子	3
5	May	May > Jan → May > Mar（第三字符 y>r）→ Mar 的右孩子	3
6	June	June > Jan（第二字符 u>a）→ June < Mar → Mar 的左孩子	3
7	July	July > Jan → July < Mar → July < June（l<n）→ June 的左孩子	4
8	Aug	Aug < Jan → Aug < Feb → Aug > Apr（u>p）→ Apr 的右孩子	4
9	Sep	Sep > Jan → > Mar → > May → May 的右孩子	4
10	Oct	Oct > Jan → > Mar → > May → < Sep → Sep 的左孩子	5
11	Nov	Nov > Jan → > Mar → > May → < Sep → < Oct → Oct 的左孩子	6
12	Dec	Dec < Jan → < Feb → > Apr → > Aug → Aug 的右孩子	5

所得二叉排序树见图 2。

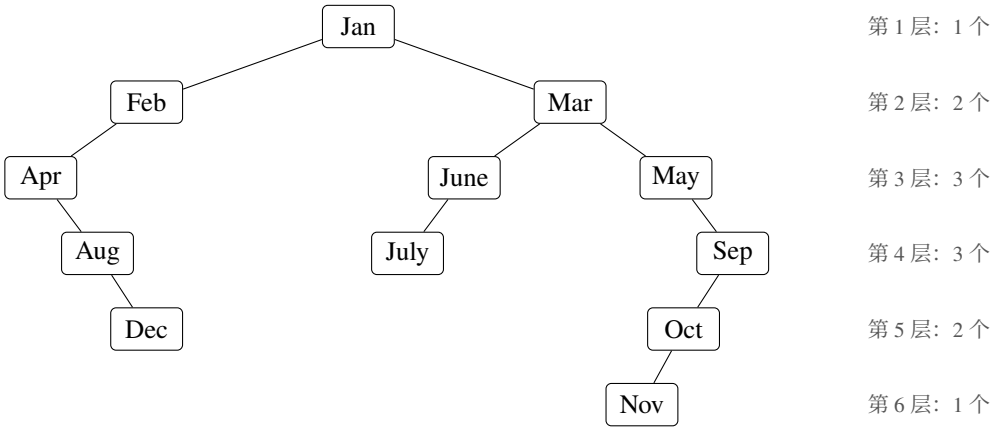


图 2: 8.1 ① 的二叉排序树。看点：插入顺序决定形状；右侧 May–Sep–Oct–Nov 形成一条“偏斜链”，拉高了树高与 ASL。

ASL 计算（解法一：按层统计） 各层结点数为 $1, 2, 3, 3, 2, 1$，故

$$\mathrm{ASL}_{\text{成功}} = \frac{1}{12}\big(1\times1+2\times2+3\times3+4\times3+5\times2+6\times1\big) = \frac{1+4+9+12+10+6}{12} = \frac{42}{12} = \frac{7}{2} = 3.5.$$

解法二：内路径长度 记内路径长度 $I = \sum$(结点深度)，深度从 0 计（根深度 0）。查找深度为 d 的结点比较 $d+1$ 次，于是 ASL $= \frac{I+n}{n}$。本题 $I = 0+1\cdot2+2\cdot3+3\cdot3+4\cdot2+5\cdot1 = 30$，ASL $= \frac{30+12}{12} = 3.5$。

方法比较 解法一直观，适合考场画图后直接数；解法二把 ASL 与“路径长度”这一树的整体量联系起来，是分析随机 BST 平均性能（$\approx 2\ln n$ 量级）的标准工具，推广性更好。二者本质相同：$\sum$(层次) $= \sum$(深度 $+1$) $= I + n$。

最终答案

二叉排序树如图 2；$\boxed{\mathrm{ASL}_{\text{成功}} = \frac{42}{12} = 3.5}$

结果检验 (1) 结点总数 $1+2+3+3+2+1 = 12$ ✓；(2) 中序遍历应为有序序列：Apr, Aug, Dec, Feb, Jan, July, June, Mar, May, Nov, Oct, Sep ✓（与排序结果一致）；(3) 已用程序模拟插入，层次与 ASL 完全一致。

② 有序表折半查找的 ASL

考查知识点 折半查找（二分查找）；折半查找判定树。

已知条件 先排序：

$$\underset{1}{\text{Apr}},\ \underset{2}{\text{Aug}},\ \underset{3}{\text{Dec}},\ \underset{4}{\text{Feb}},\ \underset{5}{\text{Jan}},\ \underset{6}{\text{July}},\ \underset{7}{\text{June}},\ \underset{8}{\text{Mar}},\ \underset{9}{\text{May}},\ \underset{10}{\text{Nov}},\ \underset{11}{\text{Oct}},\ \underset{12}{\text{Sep}}$$

解题思路 折半查找每次与区间中点 $mid = \lfloor (low + high)/2 \rfloor$ 比较，然后进入左半或右半。把“第几次比较到哪个元素”画成一棵二叉树，就是**判定树**：根是第一次比较的元素，某元素所在层次 = 找到它的比较次数。

定义：折半查找判定树

对区间 $[low, high]$ 递归构造：根为 mid，左子树为 $[low, mid-1]$ 的判定树，右子树为 $[mid+1, high]$ 的判定树。n 个结点的判定树前 $\lfloor \log_2 n \rfloor$ 层是满的，最后一层不满，高度 $h = \lfloor \log_2 n \rfloor + 1$。
使用条件：表有序且顺序存储（可随机访问中点）。本题“先排序构成有序表”恰好满足。

完整推导 区间 $[1,12]$：$mid=6$（July）；左 $[1,5]$：$mid=3$（Dec）；右 $[7,12]$：$mid=9$（May）。$[1,2]$：$mid=1$（Apr），其右为 $[2,2]$（Aug）；$[4,5]$：$mid=4$（Feb），其右 Jan；$[7,8]$：$mid=7$（June），其右 Mar；$[10,12]$：$mid=11$（Oct），左 Nov、右 Sep。判定树见图 3。

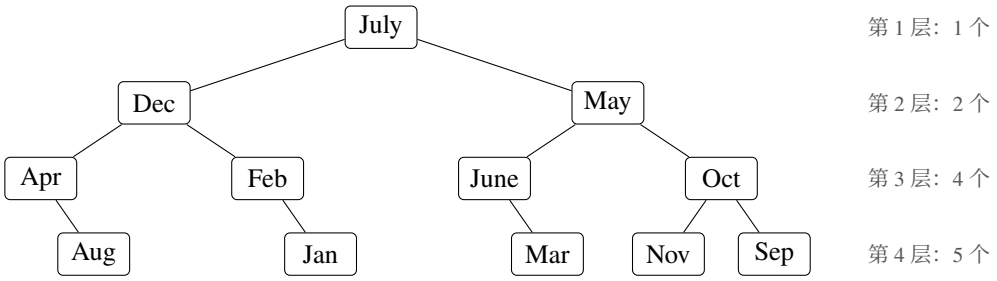


图 3: 12 个元素的折半查找判定树。看点：前 3 层是满的（1+2+4=7 个），其余 5 个落在第 4 层，所以任何元素最多比较 4 次。

ASL 计算（解法一：按判定树分层）

$$\mathrm{ASL}_{\text{成功}}=\frac{1}{12}\big(1\times1+2\times2+3\times4+4\times5\big)=\frac{1+4+12+20}{12}=\frac{37}{12}\approx3.08.$$

解法二：公式法（不画树） 判定树前 $k=\lfloor\log_2 12\rfloor=3$ 层是满的，共 $2^3-1=7$ 个结点，第 j 层有 2^{j-1} 个；剩余 $12-7=5$ 个都在第 4 层。故

$$\sum c_i=\sum_{j=1}^{3}j\cdot2^{j-1}+4\times5=(1+4+12)+20=37,\qquad \mathrm{ASL}=\frac{37}{12}.$$

方法比较 画树直观、可同时回答“查某个元素比较几次”；公式法快，且揭示了 ASL 只依赖“各层结点数”。若题目改用 $mid=\lceil(low+high)/2\rceil$，树的形状会变，但每层结点数仍为 $1,2,4,5$，ASL 不变（已用程序验证两种取法均得 $37/12$）。

最终答案

判定树如图 3；$\boxed{\mathrm{ASL}_{\text{成功}}=\dfrac{37}{12}\approx3.08}$

结果检验 最大比较次数 $=\lfloor\log_2 12\rfloor+1=4$，与树高一致 ✓；$37/12<42/12$，有序表折半查找优于①中“偏斜”的 BST，符合直觉 ✓。

③ 平衡二叉排序树（AVL 树）及其 ASL

考查知识点 平衡因子；最小不平衡子树；LL/RR/LR/RL 四种旋转。

已知条件 插入顺序与①相同；每插入一个结点后都要保持平衡。

解题目标 画出最终 AVL 树并求 ASL。

解题思路 仍按 BST 规则插入；插入后从新结点沿路径向上找**第一个**平衡因子绝对值变为 2 的结点 A（它是“最小不平衡子树”的根）；看新结点相对 A 走的前两步方向（左左 / 右右 / 左右 / 右左），选择对应旋转。旋转只改动局部三个结点的连接，保持中序序列不变、子树高度恢复为插入前的值，因此上层祖先自动恢复平衡。

定义与定理：AVL 树与旋转

平衡因子 $\mathrm{BF}(x) = h_L(x) - h_R(x)$（左子树高减右子树高）。AVL 树是每个结点都满足 $\mathrm{BF} \in \{-1, 0, 1\}$ 的 BST。

四种调整（A 为最小不平衡子树根，B 为 A 在插入路径上的孩子，C 为 B 在插入路径上的孩子）：

- LL 型（插在 A 左孩子的左子树）：以 B 为轴**右旋**一次；
- RR 型（插在 A 右孩子的右子树）：以 B 为轴**左旋**一次；
- LR 型：先对 B 左旋、再对 A 右旋，结果 C 成为新根；
- RL 型：先对 B 右旋、再对 A 左旋，结果 C 成为新根。

为什么有效：旋转只是重新选择“三者中哪个当根”，三者及其子树的中序次序不变，所以仍是 BST；并且旋转后该子树高度恢复为插入前的高度，因此一次插入最多只需一次（单或双）旋转。

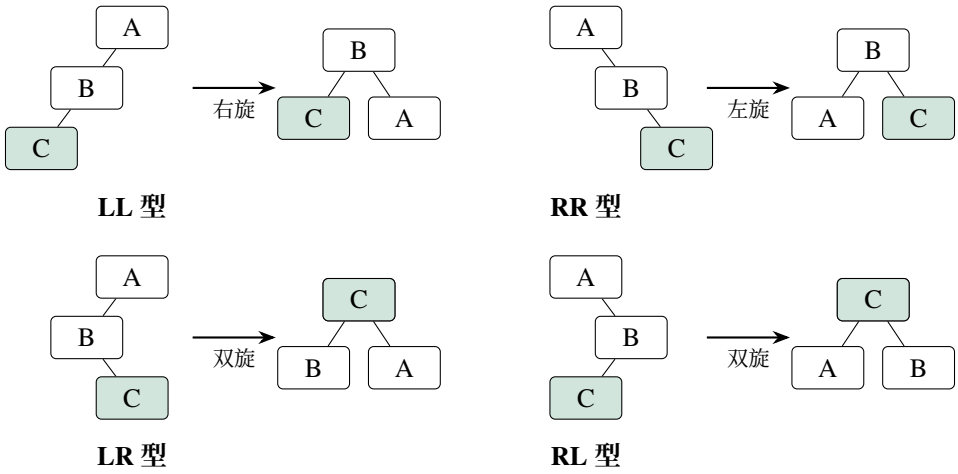


图 4: AVL 的四种调整（最简形式，绿色为“插入路径上的第三个结点”）。看点：无论哪种，最后都是“中间大小的那个结点当根”，中序次序不变。

完整推导 逐个插入并检查平衡（BF 写在结点后）：

(1) Jan、Feb、Mar：Feb 为 Jan 左孩子、Mar 为右孩子，Jan 的 $\mathrm{BF} = 0$。

(2) Apr：插在 Feb 左侧，Feb: $\mathrm{BF} = 1$，Jan: $\mathrm{BF} = 1$，平衡。

(3) May：插在 Mar 右侧，Mar: -1，Jan: 0，平衡。

(4) June：插在 Mar 左侧（June < Mar），Mar: 0，平衡。

(5) July：June 的左孩子。June: 1，Mar: 1，Jan: 左高 2、右高 3，$\mathrm{BF} = -1$，平衡。

(6) **Aug**：Aug < Jan，< Feb，> Apr，成为 Apr 的右孩子。向上检查：Apr: -1；Feb: 左高 2、右高 0，**BF** $= 2$，Feb 是最小不平衡子树的根。插入路径 Feb → 左（Apr）→ 右（Aug），属 **LR 型**：Aug 成为该子树的根，Apr 为其左孩子、Feb 为其右孩子（图 5(a)）。调整后此子树高度恢复为 2，Jan 的 $\mathrm{BF} = 2 - 3 = -1$，平衡。

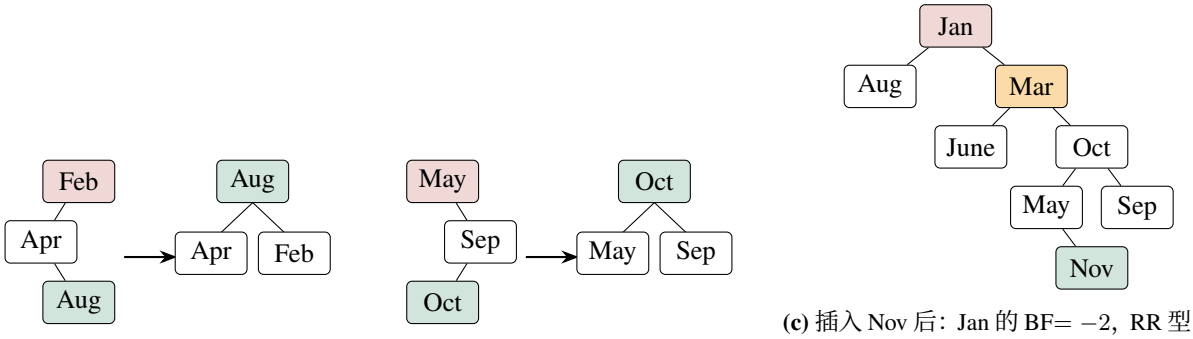
(7) Sep：May 的右孩子。May: -1，Mar: $2 - 2 = 0$，Jan: -1，平衡。

(8) **Oct**：Oct > Jan，> Mar，> May，< Sep，成为 Sep 的左孩子。Sep: 1；May: 左高 0、右高 2，**BF** $= -2$。路径 May → 右（Sep）→ 左（Oct），属 **RL 型**：Oct 成为新根，May 为其左孩子、Sep 为右孩子（图 5(b)）。子树高度恢复为 2，Mar: 0，Jan: -1，平衡。

(9) **Nov**：Nov > Jan，> Mar，< Oct（N<O），> May，成为 May 的右孩子。自下而上：May: -1；Oct: 左高 2、右高 1，$\mathrm{BF} = 1$；Mar: 左高 2（June–July）、右高 3，$\mathrm{BF} = -1$；Jan: 左高 2（Aug 子树）、右高 4，**BF** $= -2$。所以最小不平衡子树的根是 **Jan**（而不是 Oct 或 Mar）。路径 Jan → 右（Mar）→ 右（Oct），属 **RR 型**：以 Mar 为轴左旋，Mar 成为整棵树的根，Jan 成为 Mar 的左孩子，Mar 原来的左子树（June 子树）改挂为 Jan 的右子树（图 5(c)）。

(10) Dec：Dec < Mar，< Jan，> Aug，< Feb，成为 Feb 的左孩子。Feb: 1，Aug: -1，Jan: $3 - 2 = 1$，

Mar: $4-3=1$，全部平衡，不需调整。



(a) 插入 Aug：LR 型　**(b)** 插入 Oct：RL 型　**(c)** 插入 Nov 后：Jan 的 BF$=-2$，RR 型，以 Mar 为轴左旋（Aug、June 只画子树根）

图 5: 构造 AVL 树过程中的三次旋转。看点：红色结点是“最小不平衡子树”的根，必须从新插入结点向上**第一个**失衡的祖先处调整。

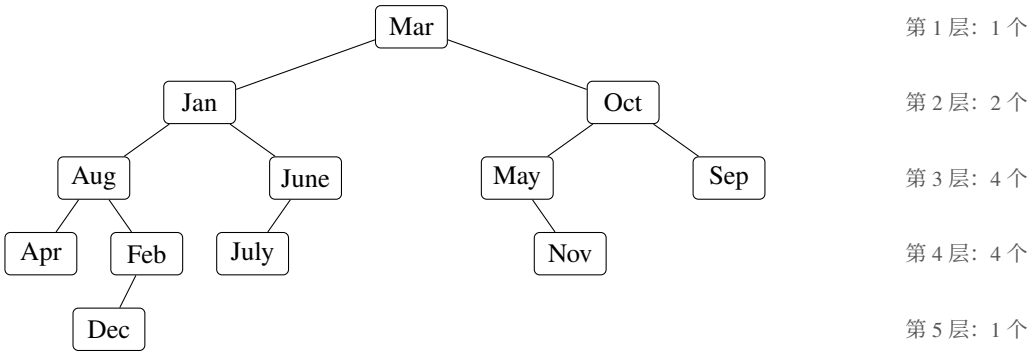


图 6: 8.1 ③ 最终得到的平衡二叉树。看点：所有结点 $|\mathrm{BF}|\le 1$，高度 5，比图 2 少一层。

ASL 计算　由图 6，各层结点数为 $1,2,4,4,1$：

$$\mathrm{ASL}_{\text{成功}}=\frac{1}{12}\big(1\times 1+2\times 2+3\times 4+4\times 4+5\times 1\big)=\frac{1+4+12+16+5}{12}=\frac{38}{12}=\frac{19}{6}\approx 3.17.$$

最终答案

平衡二叉树如图 6（旋转过程：插入 Aug 时 LR 型、插入 Oct 时 RL 型、插入 Nov 时 RR 型）；

$$\boxed{\mathrm{ASL}_{\text{成功}}=\frac{38}{12}=\frac{19}{6}\approx 3.17}$$

结果检验　(1) 逐结点核验 BF：Mar $(4-3=1)$，Jan $(3-2=1)$，Oct $(2-1=1)$，Aug $(1-2=-1)$，June (1)，May (-1)，Feb (1)，其余叶子 0，全部合法 ✓；(2) 中序序列仍为 Apr, Aug, Dec, Feb, Jan, July, June, Mar, May, Nov, Oct, Sep ✓；(3) 用程序实现 AVL 插入独立验证，三次旋转类型与位置、最终 ASL 均一致 ✓；(4) 三者比较 $37/12<38/12<42/12$：AVL 接近最优的折半判定树，远好于普通 BST，合理。

易错点

- **找错失衡结点**：插入 Nov 时，Oct 的 BF $=1$、Mar 的 BF $=-1$ 都合法，真正失衡的是 Jan。若误在 Oct 或 Mar 处旋转，得到的树不是 AVL。**自检**：旋转前从新结点逐层向上写出每个祖先的 BF，第一个出现 ± 2 的才是 A。
- **LR/RL 判断错**：型别看的是“从 A 出发、朝新结点走的前两步”，不是看新结点是左孩子还是右孩子。
- **旋转后丢子树**：RR 左旋时，轴 Mar 原来的左子树（June–July）必须改挂到 Jan 的右边，否

则丢结点或破坏有序性。**自检**：旋转后结点总数不变、中序序列不变。

【第 8.2 题】

考查知识点 m 阶 B 树的定义；插入时的结点分裂；删除时的“向兄弟借”（旋转/借位）与“合并”。

已知条件 3 阶 B 树如图 1；依次执行：插入 90、插入 25、插入 45、删除 60（每一步都在上一步结果上进行）。

解题目标 画出 4 个中间/最终结果。

定义：m 阶 B 树（本题 $m = 3$）

(1) 每个结点至多 m 棵子树、至多 $m-1$ 个关键字；(2) 根若不是叶子，至少 2 棵子树；(3) 除根外的非终端结点至少 $\lceil m/2 \rceil$ 棵子树，即至少 $\lceil m/2 \rceil - 1$ 个关键字；(4) 所有叶子（失败结点）在同一层，即树“完美等高”；(5) 结点内关键字有序，且子树关键字夹在相邻关键字之间。

代入 $m = 3$：每个结点关键字个数 $\in \{1, 2\}$，子树数 $\in \{2, 3\}$——这就是“2-3 树”。

插入规则：关键字总是插入到最底层的某个非终端结点；若结点关键字数变为 $m = 3$（上溢），则以中间关键字 $k_{\lceil m/2 \rceil}$（这里是第 2 个）为界一分为二，中间关键字上移到父结点；若父结点也上溢则继续分裂，直到根分裂使树长高一层。

删除规则（被删关键字在最底层结点时）：删除后若关键字数仍 $\geq \lceil m/2 \rceil - 1 = 1$，结束；否则（变成 0 个）：若相邻兄弟关键字数 > 1，则**借**：父结点中夹在二者之间的关键字下移到本结点，兄弟中最靠近的关键字上移到父结点；若相邻兄弟都只有 1 个关键字，则**合并**：本结点、兄弟与父结点中的分隔关键字合成一个结点，父结点少一个关键字，可能继续向上调整。

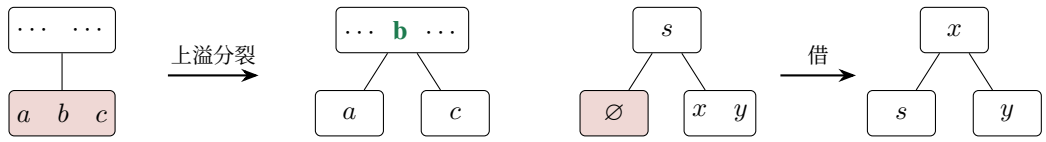


图 7: 3 阶 B 树的两个基本动作。看点：分裂时“中间关键字上去”；借位时“父结点关键字下来、兄弟关键字上去”，像一次旋转。

解题思路 每一步先按“查找”定位到最底层结点，再检查关键字数是否越界（> 2 上溢、< 1 下溢），对应采用分裂或借/合并（图 7）。

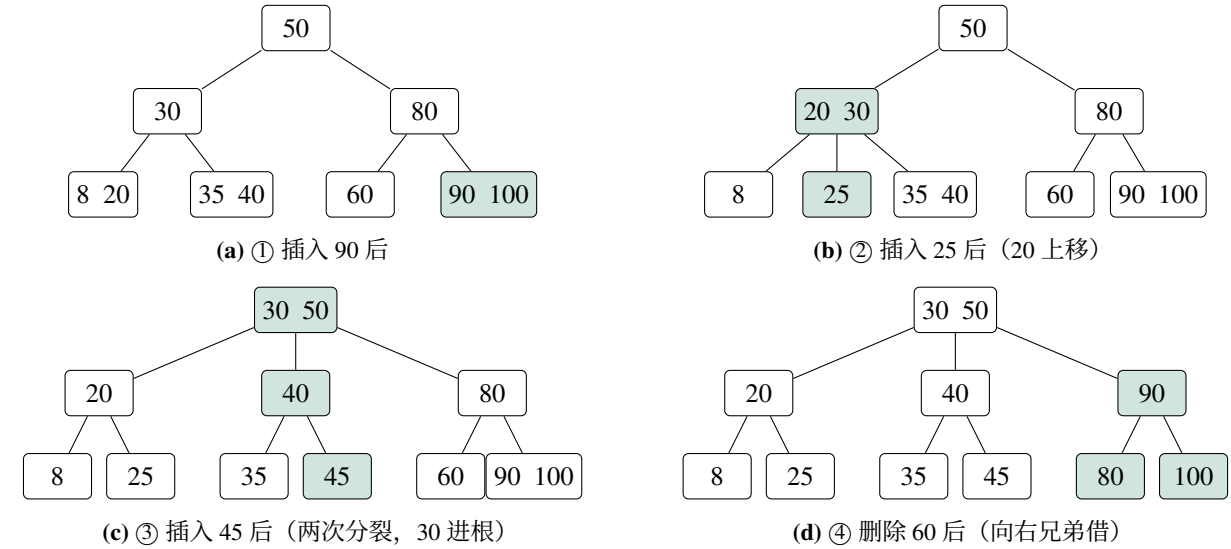
完整推导

① **插入 90**：$90 > 50$ 走右；$90 > 80$ 走 80 的右孩子 $[100]$。插入后 $[90, 100]$，2 个关键字 ≤ 2，不需分裂。

② **插入 25**：$25 < 50$ 走左；$25 < 30$ 走 $[8, 20]$。插入后 $[8, 20, 25]$ 有 3 个关键字，上溢。以中间关键字 20 分裂：$[8]$ 与 $[25]$，20 上移到父结点 $[30]$，父结点变为 $[20, 30]$（2 个关键字，合法），其三个孩子依次为 $[8], [25], [35, 40]$。

③ **插入 45**：$45 < 50$ 走左；$45 > 30$ 走 $[35, 40]$，插入后 $[35, 40, 45]$ 上溢。以 40 分裂为 $[35]$、$[45]$，40 上移，父结点变为 $[20, 30, 40]$，又上溢；再以中间关键字 30 分裂父结点为 $[20]$ 与 $[40]$，30 上移到根，根变为 $[30, 50]$。此时根有 3 个孩子：$[20]$（孩子 $[8], [25]$）、$[40]$（孩子 $[35], [45]$）、$[80]$（孩子 $[60], [90, 100]$）。树高未变（因为根没有上溢）。

④ **删除 60**：60 在最底层结点 $[60]$，删除后该结点关键字数为 0，下溢。它只有右兄弟 $[90, 100]$，有 2 个关键字（有余），可以**借**：父结点中分隔二者的 80 下移到本结点成为 $[80]$，兄弟中最小的 90 上移到父结点，父结点变为 $[90]$，兄弟变为 $[100]$。



(a) ① 插入 90 后　**(b)** ② 插入 25 后（20 上移）

(c) ③ 插入 45 后（两次分裂，30 进根）　**(d)** ④ 删除 60 后（向右兄弟借）

图 8: 8.2 各步结果（绿色为本步发生变化的结点）。看点：所有最底层结点始终在同一层；插入只会让树“向上长”。

解法二（2-3 树视角） 3 阶 B 树就是 2-3 树。可把“3 个关键字的结点”看作临时的 4-结点：把临时 4-结点的中间键“推”给父亲，左右两键各自成为 2-结点；若父亲也成了临时 4-结点就继续向上推。用这种视角重做 ②③，每一步得到的树与上面完全相同。两种描述本质一致：B 树语言强调“关键字个数上下限”，2-3 树语言强调“2-结点/3-结点的局部变换”，后者在 Sedgewick《Algorithms》等国外教材中常用，并直接通向红黑树。

最终答案

四步结果依次如图 8(a)–(d)。最终结果：
根结点 [30, 50]；　第二层 [20]、[40]、[90]；　最底层 [8]、[25]、[35]、[45]、[80]、[100]。

结果检验 (1) 每个结点关键字数都在 $\{1, 2\}$ 内，子树数 = 关键字数 + 1 ✓；(2) 所有最底层结点同层 ✓；(3) 中序序列 8, 20, 25, 30, 35, 40, 45, 50, 80, 90, 100 有序且恰为原有 8 个关键字加 3 个插入、减 1 个删除，共 11 个 ✓；(4) 用程序实现的 3 阶 B 树逐步核对，4 步结果一致 ✓。

易错点

- **阶数理解错**：“3 阶”指每结点**至多 3 棵子树**（2 个关键字），不是 3 个关键字。若误以为可放 3 个关键字，②③ 都不会分裂。
- **分裂后漏了“继续向上检查”**：③ 中 40 上移后父结点 $[20, 30, 40]$ 同样上溢，必须再分裂一次。
- **删除 60 时直接合并**：只有相邻兄弟关键字数都等于下限时才合并。由于 ① 已使右兄弟变为 $[90, 100]$，这里应“借”。若忘了 ① 的结果，会错误地合并成 $[80, 100]$，进而引起上层连锁调整。
- **借的方向错**：向右兄弟借时，上移的是右兄弟的**最小**关键字（90），下移的是父结点中的分隔关键字（80）；不能把 90 直接移到被删结点（那样会使 $80 < 90$ 的次序关系错乱）。**自检**：调整后做一次中序遍历是否有序。

【第 8.3 题】

考查知识点 除留余数法散列函数；开放定址法中的线性探测；查找成功 ASL；查找失败的判定条件。

已知条件 散列表地址 $0 \sim 17$，表长 $m = 18$；$H(key) = key \bmod 16$（值域 $0 \sim 15$）；线性探测：

$$H_i = \big(H(key) + d_i\big) \bmod m, \qquad d_i = 0, 1, 2, \ldots, \quad m = 18.$$

关键字依次为 10, 24, 32, 17, 31, 30, 46, 47, 40, 63, 49，共 $n = 11$ 个。

定义：线性探测法

发生冲突（目标地址已被占用）时，依次探查下一个地址 $H+1$, $H+2$, …，到表尾后**绕回表头**继续，直到找到空单元。取模用的是**表长** $m = 18$，而不是散列函数里的 16：散列函数只决定“起点”，探测在整张表 $0 \sim 17$ 上循环。

查找：按与插入完全相同的探测序列比较；遇到相等则成功，遇到空单元则失败（因为若该关键字存在，插入时一定会停在这个空单元或之前）。

解题思路 按输入顺序逐个计算 $H(key)$，若被占用就往后挪，记录最终地址和“探测次数”（= 查找成功时的比较次数）。

完整推导

序	key	$H(key) = key \bmod 16$	探测地址序列	说明	比较次数
1	10	10	10	空，放入	1
2	24	8	8	空，放入	1
3	32	0	0	空，放入	1
4	17	1	1	空，放入	1
5	31	15	15	空，放入	1
6	30	14	14	空，放入	1
7	46	14	14, 15, 16	14、15 被占	3
8	47	15	15, 16, 17	15、16 被占	3
9	40	8	8, 9	8 被占	2
10	63	15	15, 16, 17, 0, 1, 2	17 之后绕回 0	6
11	49	1	1, 2, 3	1、2 被占	3

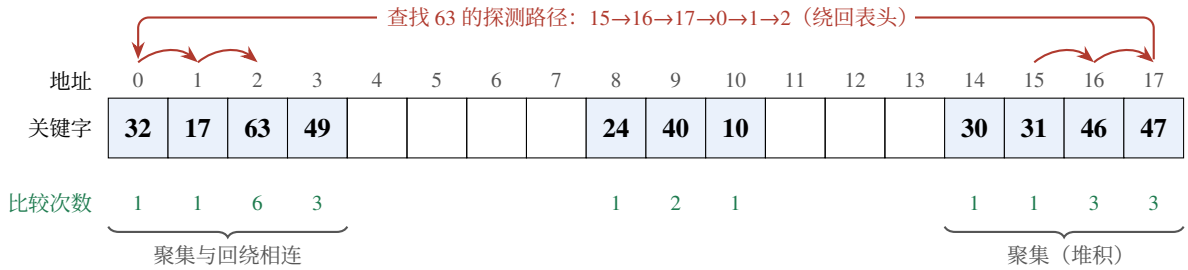


图 9: 8.3 所得散列表。看点：地址 14–17 与 0–3 首尾相接形成一大段连续占用（“一次聚集”），落在其中的关键字都要多次探测。

① 示意图 见图 9：

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
关键字	32	17	63	49					24	40	10				30	31	46	47

② **查找 63**　$H(63) = 63 \bmod 16 = 15$。依次探测地址 15, 16, 17, 0, 1, 2，即依次与 **31, 46, 47, 32, 17, 63** 比较，第 6 次比较成功。

③ **查找 60**　$H(60) = 60 \bmod 16 = 12$。地址 12 为空单元，立即判定查找失败，**不需要与任何关键字比较**。（若教材约定把“与空单元的判空”也计为一次比较，则记为 1 次比较、0 个关键字。）

④ **查找成功的 ASL**

$$\mathrm{ASL}_{\text{成功}} = \frac{1}{11}\Big(\underbrace{1+1+1+1+1+1}_{10,24,32,17,31,30} + \underbrace{3}_{46} + \underbrace{3}_{47} + \underbrace{2}_{40} + \underbrace{6}_{63} + \underbrace{3}_{49}\Big) = \frac{23}{11} \approx 2.09.$$

最终答案

① 表如图 9；② 依次与 31, 46, 47, 32, 17, 63 比较（6 次）；③ 地址 12 为空，不与任何关键字比较即失败；④ $\boxed{\mathrm{ASL}_{\text{成功}} = \dfrac{23}{11} \approx 2.09}$

结果检验　(1) 表中 11 个非空单元恰好对应 11 个关键字，无重复 ✓；(2) 对每个关键字从 $H(key)$ 起向后数到其所在位置（含回绕），距离 + 1 即比较次数，与上表一致 ✓；(3) 程序模拟一致 ✓；(4) 量级检查：装填因子 $\alpha = 11/18 \approx 0.61$，Knuth 的线性探测近似公式给出成功查找约 $\frac{1}{2}\big(1 + \frac{1}{1-\alpha}\big) \approx 1.79$ 次，本题 2.09 略大，原因是 14, 15 两个地址集中了 5 个关键字的起点，聚集偏重，合理。

易错点

- **回绕时用错模数**：探测公式取模的是表长 18。若误用 $(H+i) \bmod 16$，地址 16、17 永远探测不到，46 会被错误地放到 $14 \to 15 \to 0 \to 1 \to 2$ 的地址 2，整张表全错。**自检**：散列函数值域（$0 \sim 15$）与地址空间（$0 \sim 17$）是两回事。
- **查找 60 的回答**：失败的标志是遇到空单元。60 的起点 12 本身为空，比较的关键字序列为空，不要写成“与 10 比较”之类。
- **ASL 的分母**：查找成功的 ASL 分母是关键字个数 11，不是表长 18，也不是散列函数值域大小 16（后者是查找失败 ASL 的分母，见第三部分）。

3 第三部分 专题拓展

开篇：题目分类与筛选

综合知识重要度、考试频率、方法代表性与易错度，对本次作业分类如下（同一题可属多类）：

类别	题目	入选理由
A 重点核心题	8.1③（AVL 构造）、8.2（B 树插删）、8.3（线性探测散列）	三类动态查找结构的核心操作，几乎每年必考
B 典型基础题	8.1①（BST 构造）、8.1②（折半判定树）	ASL 计算的两种母题
C 易错题	8.1③（插入 Nov 的失衡点）、8.2④（借还是合并）、8.3②③（回绕与空单元）	计算不难但概念陷阱集中
D 综合题	8.1 整体（同一数据三种组织方式的 ASL 对比）	体现“结构决定效率”

3.1 第一章 本次作业知识体系总览

本次三道题都在回答同一个问题：**给定一批关键字，怎样组织它们，才能让“找”尽量快、并且在插入删除后依然快？**答案分成三条路线（图 10）：

- **比较 + 有序**：有序表折半查找（静态）→ 二叉排序树（动态，但可能退化）→ 平衡二叉树（动态且有高度保证）→ B 树（多路平衡，面向磁盘）。
- **计算地址**：散列表，不靠比较大小，而是“算出”位置，代价是冲突处理。
- **统一的度量**：平均查找长度 ASL，即比较次数的期望。

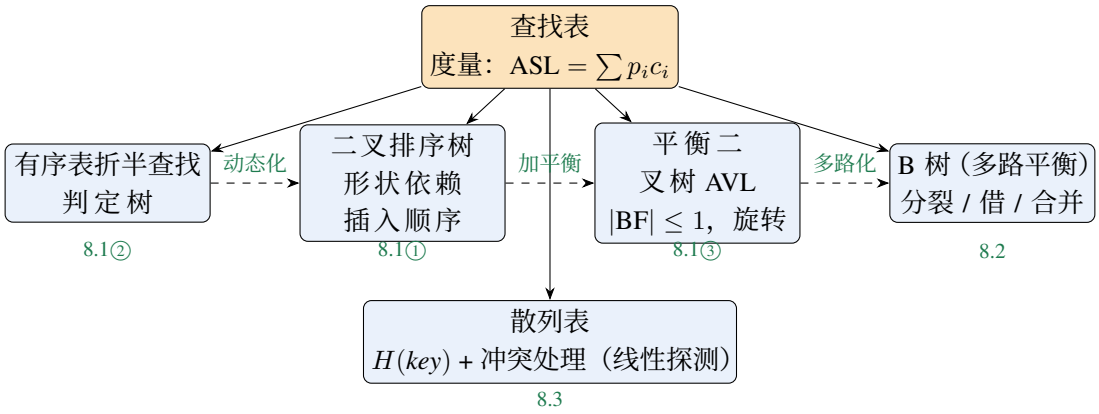


图 10: 本次作业的知识地图。看点：前四种结构沿“静态 → 动态 → 平衡 → 多路”一路演化，散列表是另一条完全不同的路线；绿色标注为对应原题。

知识点	核心概念	典型公式/定理	对应题号	常见用途
ASL	比较次数的期望	ASL $= \sum p_i c_i$	全部	比较查找结构效率
二叉排序树	左小右大、叶子插入	中序遍历有序	8.1①	动态集合、树排序
折半查找	判定树	$h = \lfloor \log_2 n \rfloor + 1$	8.1②	有序数组查找
AVL 树	平衡因子、最小不平衡子树	高度 $O(\log n)$；四种旋转	8.1③	内存中的有序字典
B 树	多路、关键字数上下限、等高	高度 $\le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$	8.2	数据库与文件系统索引
散列表	散列函数、冲突、装填因子	线性探测 $H_i = (H + i) \bmod m$	8.3	字典、集合、缓存

3.2 第二章 重点题深度回顾

8.1①②：“按层求和”母题

为什么重要：它是所有基于比较的查找结构求 ASL 的共同模型。**训练的能力**：把“查找过程”转化为“树中路径”，再转化为“层次之和”。**母题结构**：给定 n 个关键字与某种组织规则 → 构造一棵树（BST / 判定树 / AVL）→ ASL $=\frac{1}{n}\sum_{\ell}\ell\cdot(\text{第}\ell\text{层结点数})$。换数字、换成单词、换插入顺序，只是树的形状变了，方法不变。**识别信号**：“等概率”“查找成功”“二叉排序树/判定树/平衡二叉树”。

8.1③：“插入—回溯—定位—旋转”母题

为什么重要：AVL 是第一个自平衡二叉查找树，后续红黑树、伸展树等都沿用“旋转保持中序不变”这一思想。**训练的能力**：条件识别（谁失衡）、分类讨论（四型）、局部变换的不变量（中序不变、高度恢复）。**母题结构**：给定插入序列 → 每次插入后自底向上找第一个 $|\text{BF}|=2$ 的祖先 → 由前两步方向定型 → 旋转。**识别信号**：“平衡二叉树”“AVL”“依次插入”“平衡因子为 0 的结点个数”“旋转类型”。

8.2：“上溢分裂 / 下溢借合”母题

为什么重要：B 树（及其变体 B^+ 树）是数据库和文件系统中最常用的索引结构。**训练的能力**：在“关键字数上下限”这一约束下进行局部修复，并理解修复如何向上传播。**母题结构**：m 阶 B 树 + 操作序列 → 每步检查是否越界（$>m-1$ 上溢；非根 $<\lceil m/2\rceil-1$ 下溢）→ 分裂 / 借 / 合并 → 可能连锁到根。**识别信号**：“m 阶 B 树”“依次插入/删除”“画出各步结果”“结点最少/最多关键字数”。

8.3：“取余定位 + 线性探测”母题

为什么重要：散列表是工程中使用最频繁的查找结构之一。**训练的能力**：严格执行探测规则、区分“地址空间”与“散列函数值域”、区分查找成功与失败的计数口径。**母题结构**：$H(key)=key \bmod p$，表长 $m\ (\geq p)$，线性探测 → 构表 → 查找某关键字的比较序列 → 成功/失败 ASL。**识别信号**：“地址范围”“哈希函数”“线性探测”“装填因子”“依次与哪些关键字比较”。

3.3 第三章 重点题举一反三

8.1 的变式

变式 1-1（条件变化：由“给序列求树”改为“给树求序列”）

用同样 12 个月份，找出一种插入顺序，使普通二叉排序树恰好等于图 3 的折半查找判定树，并求其 ASL。

考查点：BST 的形状由“每个结点相对其祖先的插入先后”决定。**思路**：只要每个结点都在它的所有祖先之后插入，它就会沿着与判定树相同的路径落到同一位置；最简单的做法是按判定树的**层序**插入。**解**：按层序：July, Dec, May, Apr, Feb, June, Oct, Aug, Jan, Mar, Nov, Sep。逐个插入可验证每个关键字落点与图 3 相同（例如 Jan：Jan < July → 走左到 Dec；Jan > Dec → 走右到 Feb；Jan > Feb → 成为 Feb 的右孩子，与判定树一致 ✓）。ASL $=\frac{37}{12}$。**与原题比较**：原题是“正向模拟”，这里是“逆向构造”；同一套插入规则，答案不唯一（如 July, May, Dec, …也可）。程序验证该序列所得 BST 层次和为 37 ✓。

变式 1-2（条件变化：有序输入 + 平衡）

把 Jan, Feb, Mar, Apr, May, June, July 这 7 个月份先按字典序排序，再按排序后的顺序依次插入 (a) 普通 BST；(b) AVL 树。分别求 ASL。

思路：有序输入是 BST 的最坏情况——每个新关键字都比已有的大，只能一直往右挂。AVL 会不断触发 RR 旋转把它“掰回来”。**解**：排序后为 Apr, Feb, Jan, July, June, Mar, May。(a) 普通 BST

退化为一条右斜链，层次为 $1,2,\ldots,7$，ASL $=\frac{1+2+\cdots+7}{7}=\frac{28}{7}=4$。(b) AVL：插入 Jan 时 Apr 失衡（RR），左旋得 Feb(Apr, Jan)；插入 July 不失衡；插入 June 时 Jan 失衡（RR），左旋得 Feb(Apr, July(Jan, June))；插入 Mar 时根 Feb 失衡（RR），左旋得 July(Feb(Apr, Jan), June(—, Mar))；插入 May 时 June 失衡（RR），左旋得 Mar(June, May)。共 4 次 RR 旋转，最终得到满二叉树

$$\text{July}\big(\text{Feb}(\text{Apr},\text{Jan}),\ \text{Mar}(\text{June},\text{May})\big),\qquad \text{ASL}=\frac{1+2\cdot 2+3\cdot 4}{7}=\frac{17}{7}\approx 2.43.$$

检验：程序模拟的旋转序列与最终树一致；$17/7<4$，体现平衡的价值。

变式 1-3（综合提升：AVL 高度上界）

含 12 个结点的 AVL 树最大可能高度是多少？8.1③ 的结果是否达到这个上界？12 个结点的二叉树 ASL 的最小可能值是多少？

公式：设 N_h 为高度 h 的 AVL 树最少结点数。最“瘦”的 AVL 树根的两棵子树高度分别为 $h-1$ 与 $h-2$，且各自也最瘦，于是

$$N_h=N_{h-1}+N_{h-2}+1,\quad N_1=1,\ N_2=2\ \Longrightarrow\ N_3=4,\ N_4=7,\ N_5=12,\ N_6=20.$$

解：$N_5=12\le 12<N_6=20$，所以 12 个结点的 AVL 树最大高度为 **5**。图 6 高度恰为 5，正好是 12 个结点所能达到的“最瘦”AVL 树（每个分支结点的 BF 都非零，可逐一核对）。ASL 最小值在“完全二叉树”时取得，各层 $1,2,4,5$，即 $\frac{37}{12}$——恰好等于折半判定树。因此 8.1③ 的 $\frac{38}{12}$ 只比理论最优多 $\frac{1}{12}$。N_h+1 正是斐波那契数（2, 3, 5, 8, 13, 21），见第六章图 13。

8.2 的变式

变式 2-1（条件变化：删除非最底层结点中的关键字）

在图 1 的原始 B 树上直接删除 50，画出结果。

思路：被删关键字在非终端结点时，用其**中序后继**（右子树中最小关键字）替代，转化为删除最底层结点中的关键字。**推导**：50 的后继是右子树最左的 60。用 60 替换根中的 50，再删除最底层的 60：[60] 变空，其唯一兄弟 [100] 只有 1 个关键字，无法借，只能**合并**：空结点 + 父结点的 80 + [100] 合成 $[80,100]$，父结点 [80] 变空（下溢）。它的兄弟 [30] 也只有 1 个关键字，再合并：[30] + 根中的 60 + 空结点合成 $[30,60]$，其孩子为 $[8,20],[35,40],[80,100]$。根变空，被删除，**树高减 1**。**答案**：根 $[30,60]$，孩子 $[8,20]$、$[35,40]$、$[80,100]$（图 11(a)）。**检验**：中序 8, 20, 30, 35, 40, 60, 80, 100 有序 ✓；程序验证一致 ✓。**与原题比较**：原题删除 60 时兄弟“富裕”可借；本题兄弟“贫穷”只能合并并连锁到根，是删除的另一主要分支。

变式 2-2（基础迁移：在原题最终结果上继续删除）

在图 8(d) 的基础上删除 8。

推导：[8] 删除后为空，兄弟 [25] 只有 1 个关键字 → 合并：$[20,25]$，父结点 [20] 变空。父结点的右兄弟 [40] 只有 1 个关键字 → 再合并：空结点 + 根中的 30 + [40] 合成 $[30,40]$，孩子为 $[20,25],[35],[45]$；根由 $[30,50]$ 变为 [50]（仍有 1 个关键字，合法），结束。**答案**：根 [50]；第二层 $[30,40]$、[90]；最底层 $[20,25],[35],[45],[80],[100]$（图 11(b)）。**检验**：所有最底层结点同层 ✓，程序验证 ✓。



(a) 变式 2-1：删除 50（树高减 1） **(b)** 变式 2-2：再删除 8（两级合并）

图 11: B 树删除中的“合并”分支。看点：合并会把父结点的关键字“拉下来”，可能一路传到根，使树变矮——这是 B 树高度减小的唯一方式。

8.3 的变式

变式 3-1（条件变化：求查找失败的 ASL）

对 8.3 所建的表，假设待查关键字的散列值等概率地落在 $0 \sim 15$，求查找失败的平均查找长度（约定“与空单元的判空比较”也计 1 次）。

思路：失败查找从起点 $j = H(key)$ 出发，一直探测到空单元为止；次数只取决于起点 j。起点只能是散列函数值域 $0 \sim 15$，所以分母是 16。**推导**：对每个起点 j 数到第一个空单元（含空单元）的探测次数：

j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
次数	5	4	3	2	1	1	1	1	4	3	2	1	1	1	9	8

例如 $j = 14$：14, 15, 16, 17, 0, 1, 2, 3 均被占，第 9 次探到空的 4。合计 47，故 $\mathrm{ASL}_{\text{失败}} = \frac{47}{16} \approx 2.94$（程序验证 ✓）。**与原题比较**：成功 ASL 的“分母是关键字数”，失败 ASL 的“分母是散列函数可能取值数”；聚集对失败查找的伤害更大（起点 14 要 9 次）。

变式 3-2（方法改变：改用链地址法）

其余条件不变，冲突改用链地址法（同义词链表，表头 $0 \sim 15$，新结点接在链尾），求查找成功的 ASL。

推导：按 $key \bmod 16$ 分组：0:{32}，1:{17, 49}，8:{24, 40}，10:{10}，14:{30, 46}，15:{31, 47, 63}。链中第 k 个结点需比较 k 次：

$$\mathrm{ASL}_{\text{成功}} = \frac{1 \times 6 + 2 \times 4 + 3 \times 1}{11} = \frac{17}{11} \approx 1.55.$$

对比：$17/11 < 23/11$。链地址法中只有**同义词**（散列值相同）才互相影响；线性探测中 63 与 32、17 并不同义却要比较，这就是“聚集”的代价。

变式 3-3（综合：开放定址表中的删除）

在 8.3 的表中删除 17（直接把地址 1 置空），再查找 49，会发生什么？应如何正确删除？

分析：$H(49) = 1$，若地址 1 被置空，查找 49 在第一步就遇到空单元，误判“49 不存在”，尽管它仍在地址 3。**正确做法**：开放定址表不能物理删除，应在地址 1 放“已删除”标记（墓碑）：查找时视其为“非空且不相等，继续探测”，插入时可复用该位置。**结论**：线性探测的正确性依赖于“探测链不被中途截断”。

3.4 第四章 易错题专题

易错 1：字符串大小判断（8.1）

错误来源：凭“月份先后”或“单词长短”比较大小。
错误示范：认为 June 在 July 之前（6 月早于 7 月）所以 June $<$ July，把 July 挂到 June 右侧。
为什么错：BST 比较的是关键字的字典序，与语义无关。June 与 July 的前缀 Ju 相同，第三字符 n $>$ l，故 July $<$ June。
正确规则：逐字符比较 ASCII 码；遇到第一个不同字符即定胜负；一方是另一方前缀时短者小。
自检：画完树后写出中序遍历，必须与 `sort` 的结果一致。

易错 2：AVL 失衡结点定位（8.1③ 插入 Nov）

错误示范：看到 Nov 插在 May 下面，就在 May–Oct 附近做一次旋转，或认为 Oct 的 BF$=1$“有点偏”而旋转 Oct。
为什么错：旋转只在 $|\mathrm{BF}|=2$ 的结点进行；May（-1）、Oct（1）、Mar（-1）都合法，真正失衡的是根 Jan（-2）。
正确方法：从新结点父亲开始逐层向上更新高度、计算 BF，第一个 ± 2 者为 A；型别由“A 到新结点的前两步”决定（这里是右、右 $\rightarrow$ RR）。
自检：旋转后再算一遍所有结点 BF，且中序序列不变。

易错 3：B 树阶数与上下限（8.2）

错误示范：“3 阶 B 树每个结点可以有 3 个关键字”，于是插入 25 后保留 $[8, 20, 25]$ 不分裂。
为什么错：m 阶指最多 m 棵子树，关键字最多 $m-1=2$ 个。
另一个常见错误：删除 60 时直接“删掉空结点”，导致 80 只剩 1 个孩子，违反“子树数 = 关键字数 + 1”。
正确规则：上限 $m-1$、非根下限 $\lceil m/2 \rceil - 1$；下溢先看相邻兄弟能否借，不能借才合并。
自检：每个结点检查 (关键字数, 子树数) 是否为 (1, 2) 或 (2, 3)；所有最底层结点是否同层。

易错 4：线性探测的回绕与计数口径（8.3）

错误示范 A：63 探到 17 后写“表满，插入失败”，或用 mod16 回绕。
错误示范 B：查找 60 时写“与 10 比较一次失败”（把 60 当作落在地址 10 附近）。
为什么错：线性探测在**整个地址空间**上循环（模表长 18），只要表未满总能找到空位；查找失败的唯一判据是遇到空单元。
正确方法：严格写出探测序列 $H, H+1, \ldots \pmod{18}$。
自检：成功查找的比较次数 = 从起点到所在位置的循环距离 + 1；所有关键字次数之和除以关键字数。

易错 5：折半判定树的 ASL（8.1②）

错误示范：直接套用近似公式 ASL $\approx \log_2(n+1) - 1 = \log_2 13 - 1 \approx 2.70$ 作为答案。
为什么错：该式是 n 较大时的近似（精确式仅在 $n = 2^h - 1$ 时成立），$n = 12$ 时与精确值 $37/12 \approx 3.08$ 相差明显。
正确方法：小规模题一律按判定树逐层计数。
自检：最大比较次数必须等于 $\lfloor \log_2 n \rfloor + 1$；各层结点数之和等于 n。

3.5 第五章 易错题举一反三

易错变式 E1（字典序陷阱）

将 Mar, May, Ma, Mars, M 依次插入空二叉排序树，求 ASL。

最易犯的错误：认为 Ma 比 Mar“少一个字母所以无法比较”或按长度比较。**正确解法**：Mar 为根；May > Mar（y>r）→ 右；Ma 是 Mar 的前缀 → Ma < Mar → 左；Mars 以 Mar 为前缀 → Mars > Mar → 右，再与 May 比：r<y → May 左；M 是 Ma 的前缀 → 一路向左，成为 Ma 的左孩子。层次：Mar 1，Ma 2，May 2，M 3，Mars 3。ASL $= \frac{11}{5} = 2.2$（程序验证 ✓）。**防错原则**：字典序 = 逐字符比较 + 前缀者小。

易错变式 E2（失衡结点不是新结点的祖父）

将 30, 20, 40, 10, 25, 22 依次插入空 AVL 树，画出结果。

最易犯的错误：插入 22 后在 25 或 20 处旋转。**正确解法**：插入 22（$20 < 22 < 25$）成为 25 的左孩子。25: BF$=1$；20: 左高 1、右高 2，BF$=-1$（合法）；30: 左高 3、右高 1，**BF**$=2$。从 30 出发前两步为“左（20）、右（25）”→ **LR 型**：25 成为新根，25 原来的左子树 22 改挂为 20 的右孩子。结果 $25\big(20(10,22),\ 30(-,40)\big)$，所有 BF 为 $0, 0, -1$ 等合法值（程序验证 ✓）。**防错原则**：A 由“第一个 $|\mathrm{BF}|=2$”决定，型别由“A 往下的两步”决定，C 的子树要按大小重新分给 A、B。

易错变式 E3（向左兄弟借）

3 阶 B 树：根 [30]，孩子 [10, 20] 与 [40]。删除 40，画出结果。

最易犯的错误：把左兄弟的 10（最小者）提上去，或把 20 直接移进被删结点。**正确解法**：[40] 删空，左兄弟 [10, 20] 有余 → 父结点 30 下移成 [30]，左兄弟**最大**关键字 20 上移，结果根 [20]，孩子 [10]、[30]。**防错原则**：借位等价于一次“旋转”：向左兄弟借，上移它的最大者；向右兄弟借，上移它的最小者——上移的必须是紧邻分隔关键字的那个，才能保持有序。

易错变式 E4（表长与模数不一致）

散列表地址 $0 \sim 12$（$m = 13$），$H(key) = key \bmod 11$，线性探测，依次插入 10, 21, 32, 43，求各自地址与 ASL。

最易犯的错误：用 $(H+i) \bmod 11$ 回绕，把 21 放到地址 0。**正确解法**：四者散列值都为 10；10 → 地址 10；21 → 10 占用 → 11；32 → 10, 11 占用 → 12；43 → 10, 11, 12 占用 → $(10+3) \bmod 13 = 0$。比较次数 1, 2, 3, 4，ASL $= \frac{10}{4} = 2.5$。**防错原则**：探测取模用表长 m；散列函数的模 p 只决定起点。

易错变式 E5（判定树计数）

有序表长度为 10，求折半查找成功的 ASL。

最易犯的错误：用 $\log_2 11 - 1$ 近似。**正确解法**：前 3 层满（7 个），第 4 层 3 个：ASL $= \frac{1+2\cdot2+3\cdot4+4\cdot3}{10} = \frac{29}{10} = 2.9$（程序验证 ✓）。**防错原则**：小 n 时按层精确计数。

3.6 第六章 核心知识系统回顾

1. 平均查找长度 ASL

是什么：比较次数的数学期望。**解决什么**：用一个数比较不同查找结构的平均效率。**为什么成立**：设查找第 i 个记录的事件概率为 p_i、比较次数为 c_i，比较次数这个随机变量的期望就是 $\sum p_i c_i$。**使用条件**：要说明是“成功”还是“失败”，以及概率分布（通常等概率）。**最易混淆**：成功与失败的分母不同。图 12 把本次作业中的各种 ASL 放在一起比较。

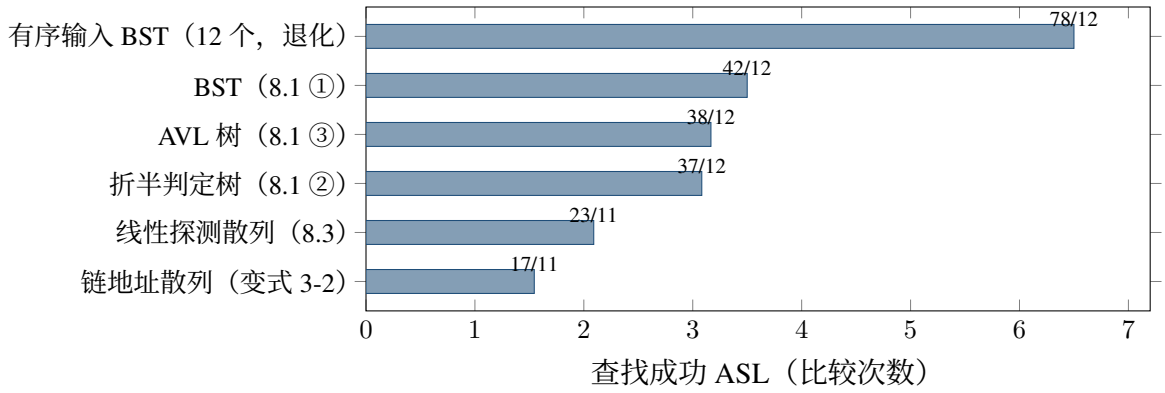


图 12: 同一类数据在不同组织方式下的查找成功 ASL。看点：平衡结构逼近最优判定树；散列表的 ASL 与 n 无直接关系，主要由装填因子与冲突处理方式决定。

2. 二叉排序树与折半查找判定树

是什么：BST 是“左小右大”的二叉树；判定树是折半查找过程画成的树，它本身也是一棵 BST，而且是最矮的那一类。**为什么成立**：中序遍历 BST 恰好得到有序序列（归纳：左子树全小于根、右子树全大于根）。**联系**：折半查找 = 在一棵“预先平衡好”的静态 BST 上查找（图 3）；BST = 可动态插入但可能失衡的折半查找（图 2）。**常见题型**：构造、求 ASL、求某关键字比较次数、判断某比较序列是否可能。

3. AVL 树

核心定理：n 个结点的 AVL 树高度 h 满足 $h < 1.44\log_2(n+2)$ 左右，即 $O(\log n)$。**为什么**：最瘦 AVL 树的结点数满足 $N_h = N_{h-1} + N_{h-2} + 1$，于是 $N_h + 1$ 是斐波那契数，按 φ^h（$\varphi \approx 1.618$）指数增长；反过来 h 只能按对数增长（图 13）。**使用条件**：每次插入/删除后都要维护 BF。**最易混淆**：“平衡”是指**每个**结点 $|\mathrm{BF}| \le 1$，不是只看根。

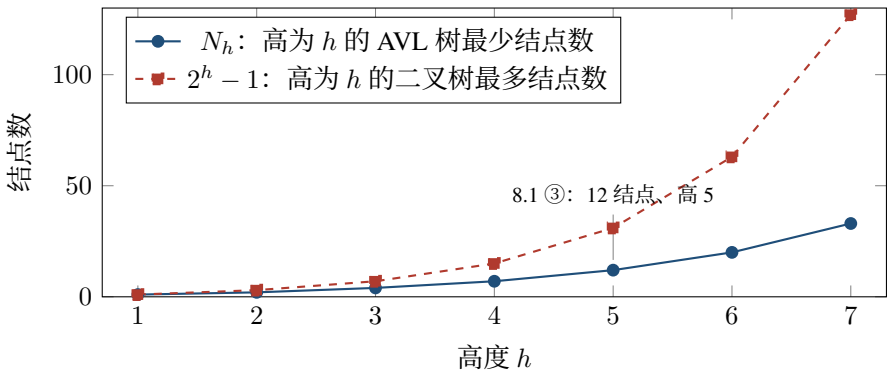


图 13: AVL 树高度与结点数的关系。看点：两条曲线都指数增长，说明 AVL 树最坏高度也只是 $\log n$ 的常数倍；8.1 ③的树恰好位于下边界上。

4. B 树

是什么：多路、等高、结点内关键字数有上下限的查找树。**解决什么**：数据放在磁盘上时，读一次磁盘（一个页）代价远大于在内存里比较几十个关键字，所以要让树“又胖又矮”，减少读盘次数。**核心公式**：含 n 个关键字的 m 阶 B 树高度（不含失败结点层）$h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$。**为什么**：非根结点至少 $\lceil m/2 \rceil$ 个孩子，按最少情形逐层计数，第 $h+1$ 层（失败结点）至少 $2\lceil m/2 \rceil^{h-1}$ 个，而失败结点数恰为 $n+1$。**联系**：3 阶 B 树 = 2-3 树；B^+ 树把全部数据放在叶子并用链表串联，便于范围查询（图 14）。



B 树：每个关键字只出现一次 **B$^+$ 树：数据全在叶子，叶子成链**

图 14: B 树与 B$^+$ 树的结构对比（示意，B$^+$ 树采用“非叶结点存子树最大关键字”的一种常见画法）。看点：B$^+$ 树非叶结点只做“路标”，叶子链表让“查找 30 到 70 之间所有记录”只需顺链扫描。

5. 散列表与线性探测

是什么：用散列函数把关键字直接映射为地址。**核心公式**：装填因子 $\alpha = n/m$；线性探测在随机散列假设下（Knuth）

$$\mathrm{ASL}_{\text{成功}} \approx \frac{1}{2}\left(1+\frac{1}{1-\alpha}\right), \qquad \mathrm{ASL}_{\text{失败}} \approx \frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right).$$

为什么：冲突会使占用单元连成片（一次聚集），新关键字落在片中任一位置都会被推到片尾，使片继续变长，所以 $\alpha \to 1$ 时代价急剧上升（图 15）。**使用条件**：上式是 m 很大时的平均性能近似，小表只能作量级参考。**最易混淆**：散列函数值域与地址空间；冲突与聚集。

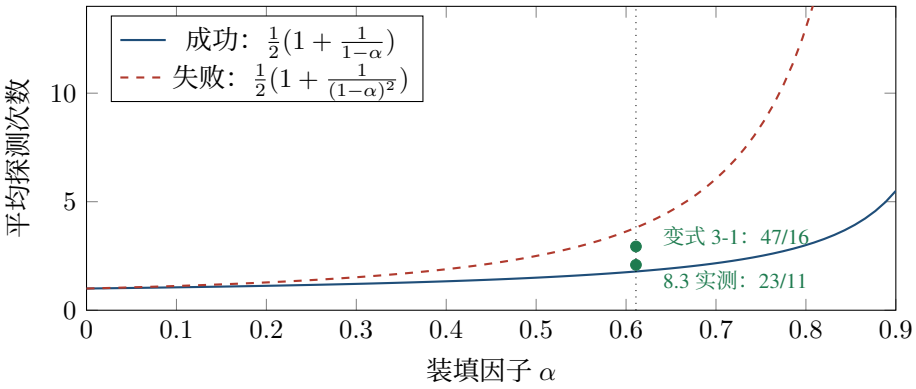


图 15: 线性探测的理论期望代价随装填因子的变化（$\alpha = 11/18 \approx 0.61$ 处为本题）。看点：α 超过 0.7 后失败查找代价陡增，这正是工程中散列表在 α 到某个阈值时就扩容的原因。

6. 相似概念辨析

概念	定义	核心区别	常见误区	判断方法
BST vs AVL	AVL 是额外满足 $\lvert BF\rvert \le 1$ 的 BST	AVL 插入后要旋转	以为“根平衡”就是 AVL	逐结点算 BF
B 树 vs B$^+$ 树	B$^+$ 树数据全在叶子	B$^+$ 非叶只做索引、叶子成链	混用二者的结点关键字数规定	看关键字是否在叶子重复出现
成功 ASL vs 失败 ASL	找到 / 确认不存在的期望比较数	分母：关键字数 vs 起点可能数	失败 ASL 也除以 n	先写清概率空间
值域 vs 地址空间	H 的取值 $0 \sim 15$ vs 表的下标 $0 \sim 17$	前者定起点，后者定回绕	用 16 取模回绕	看清“地址范围”与“哈希函数”两句
冲突 vs 聚集	两关键字散列值相同 vs 非同义词争夺同一片地址	聚集是线性探测特有的副作用	以为只有同义词才互相影响	看 63 与 32 是否同义
借 vs 合并	从兄弟取 1 个关键字 vs 与兄弟合成一个结点	借不改变父结点关键字数	兄弟有余仍合并	先看相邻兄弟关键字数

7. 知识迁移

- **本课程后续**：红黑树（与 2-3 树一一对应）、堆、哈希在图算法中的应用（如判重）。
- **高阶课程**：《数据库系统》中的 B^+ 树索引与哈希索引；《操作系统》文件系统目录索引；《算法设计与分析》中的平摊分析与随机化分析（随机 BST 的期望高度、通用散列）。
- **控制与嵌入式（本专业相关）**：实时控制程序需要可预测的最坏执行时间，平衡树给出 $O(\log n)$ 的最坏保证，而散列表只有平均 $O(1)$、最坏可达 $O(n)$——这是选型时真实存在的权衡。

3.7 第七章 学科科普：我们学的“查找”究竟是什么

这个领域研究什么。数据结构研究“数据怎样摆放，才能让需要的操作又快又省”。查找是其中最古老也最常用的操作：手机里搜联系人、编译器查变量名、数据库按学号取成绩、浏览器判断网址是否访问过，本质都是“在一堆关键字里判断某个关键字在不在、在哪里”。

它解决什么类型的问题。查找问题有两个维度：数据是静态还是动态（会不会频繁插入删除），数据在内存还是外存（一次访问的代价是纳秒级还是毫秒级）。本次作业的四种结构正好覆盖了这些组合：有序表折半（静态、内存）、BST/AVL（动态、内存）、B 树（动态、外存）、散列表（动态、内存为主，不要求有序）。

为什么本科要学。一方面它是算法分析的入门训练场：ASL 让我们第一次用“期望”去衡量程序；另一方面这些结构直接出现在你今后使用的每个标准库和数据库里——C++ 的 `std::map` 在主流实现（如 GCC 的 libstdc++）中是红黑树，`std::unordered_map` 是散列表。

在学科中的位置。

线性表、二叉树遍历 → **本次：查找结构与 ASL** → 排序、图算法中的判重与索引 → 数据库索引、文件系统、编译器符号表、缓存系统

进一步会学到什么。红黑树与跳表（另一类平衡思路）、通用散列与完美散列（用随机化对抗最坏情况）、B^+ 树与 LSM 树（现代存储引擎）、以及“缓存友好”的数据布局。

3.8 第八章 科学史：从“对半猜”到 B 树

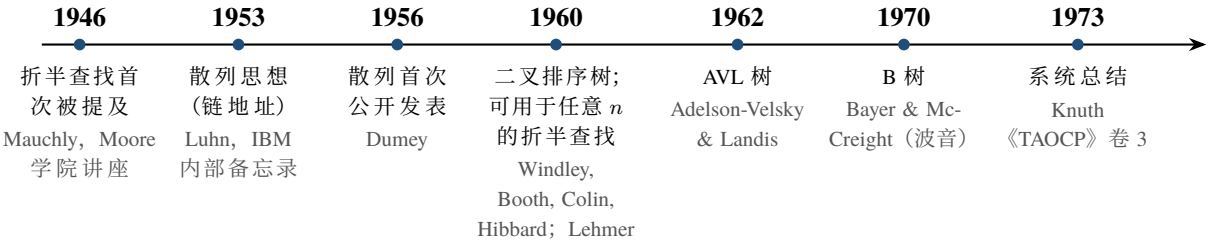


图 16: 本次作业所涉结构的关键发展时间线。看点：这些今天写在教材第 8 章里的方法，大多诞生于 1946–1973 年这短短二十多年，伴随电子计算机从无到有。

1946—1960：折半查找为什么“很简单却写了十几年”

据 Knuth《计算机程序设计艺术》卷 3 的考证，折半查找最早的公开提及来自 1946 年 John Mauchly 在宾夕法尼亚大学 Moore 学院讲座中的介绍；而早期描述只适用于长度为 2 的幂的表，直到 1960 年 D. H. Lehmer 才发表了适用于任意 n 的版本。后来 Jon Bentley 在《编程珠玑》中提到大量程序员写不对折半查找。**与本题的联系**：8.1② 中 $n = 12$ 不是 $2^k - 1$，判定树最后一层不满，“边界怎么取、区间如何收缩”正是当年困扰人们的地方。

1953—1956：散列与线性探测

Knuth 在 TAOCP 卷 3 §6.4 的历史注记中写道：散列思想似乎源于 IBM 的 H. P. Luhn，他在 1953 年 1 月的一份内部备忘录中建议使用链地址法；而“开放定址 + 线性探测”的想法被归功于同在 IBM 的 Gene Amdahl。散列首次在公开文献中描述，一般归于 Arnold I. Dumey（1956 年）。IEEE Spectrum 2018 年的史话文章也记述了 Luhn 用“桶”加速检索的思路。**与本题的联系**：8.3 使用的正是线性探测，而变式 3-2 的链地址法则更接近 Luhn 最初的方案。（注：这些早期工作多为内部文档，“谁第一个”存在多种说法，此处以 Knuth 的记述为准。）

1960—1962：二叉排序树与 AVL 树

二叉排序树在 1960 年前后被多位研究者独立提出（P. F. Windley、A. D. Booth 与 A. J. T. Colin、T. N. Hibbard 等）。人们很快发现它的形状取决于插入顺序，会像 8.1① 那样偏斜，甚至像变式 1-2 那样退化成链表。1962 年，苏联数学家 G. M. Adelson-Velsky 与 E. M. Landis 在《苏联科学院报告》发表“一种信息组织算法”，提出用高度差不超过 1 的约束加局部旋转来维持平衡，这就是以二人姓氏首字母命名的 AVL 树，也是第一种自平衡二叉查找树。Adelson-Velsky 后来还领导了国际象棋程序 Kaissa 的开发。**与本题的联系**：8.1③ 让我们亲手重演了这个发明——同一组月份，BST 的 ASL 是 3.5，加上平衡后降到 3.17。

1970：B 树——为磁盘而生

1970 年 7 月，波音科学研究实验室的 Rudolf Bayer 与 Edward McCreight 完成报告《大型有序索引的组织与维护》（1972 年发表于 *Acta Informatica*）。他们面对的现实是：索引太大，只能放在磁盘或磁鼓上，每读一页都很慢，于是设计了每个结点就是一页、多路且等高的 B 树，并用实验证明在 IBM 360/44 上可以维护十万量级关键字的索引。至于“B”代表什么，两位作者从未给出定论（Boeing、Balanced、Bayer 等说法都有）。**与本题的联系**：8.2 中“满了就分裂、中间关键字上移”“空了就借或合并”，就是这份报告所定义的维护算法的 3 阶缩影。

3.9 第九章 关联学科知识网络与跨课程迁移

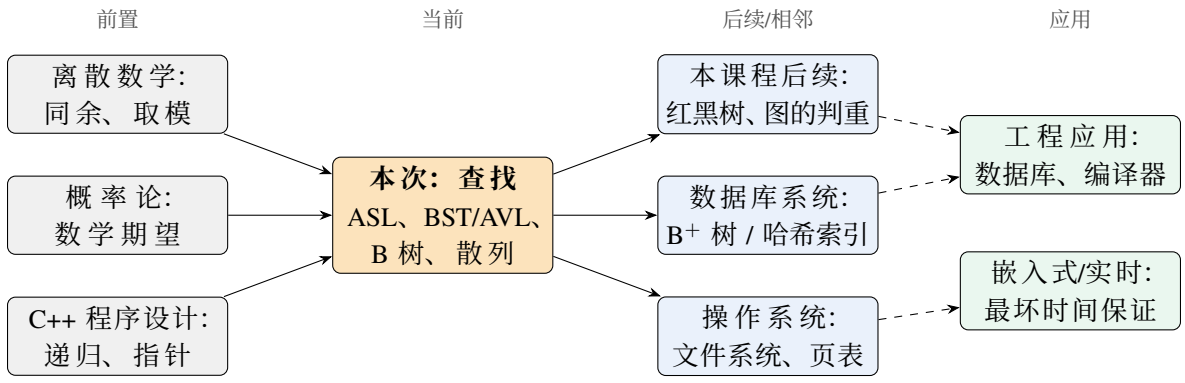


图 17: 查找相关的课程知识链。看点：左侧三门前置课分别支撑“散列函数”“ASL”“代码实现”；右侧是这些结构在后续课程中的再次出现。

当前知识点	关联课程	关联知识点	关系	为什么有关	原题	层级
ASL	概率论	离散型随机变量期望	前置·强	ASL 就是比较次数的期望	全部	必须掌握
散列函数	离散数学	同余与剩余类	前置·强	*key* mod 16 把整数划分为 16 个剩余类	8.3	必须掌握
BST/AVL	C++ 程序设计	递归、指针	前置·强	插入与旋转都靠递归和改指针实现	8.1	建议复习
AVL	本课程后续	红黑树	后续·强	红黑树可看作 2-3 树的二叉编码	8.1 ③、8.2	后续系统学习
B 树	数据库系统	B^+ 树索引	后续·强	数据库按页组织索引	8.2	后续系统学习
散列	算法分析	通用散列	后续·拓展	用随机选择散列函数避免最坏输入	8.3	拓展了解
平衡树 vs 散列	实时/嵌入式系统	最坏执行时间	应用·拓展	控制循环要求时间可预测	8.1、8.3	拓展了解

最小必要知识 1：数学期望。离散型随机变量 X 取值 x_i 的概率为 p_i，则 $E[X]=\sum p_i x_i$。现在需要它，是因为 ASL 的定义就是 E[比较次数]。掌握到能写出概率空间并计算即可。**小示例**：若 8.1② 中各月份被查概率与其天数成正比（如 Feb 为 28/365），ASL 就不再是 37/12，而是 $\sum \frac{\text{天数}_i}{365} c_i$——这正是“最优二叉查找树”（动态规划章节）要解决的问题：概率不均时，常被查的关键字应靠近根。

最小必要知识 2：同余。$a \equiv b \pmod{p}$ 当且仅当 $p \mid (a-b)$。8.3 中 $31, 47, 63$ 都 $\equiv 15 \pmod{16}$，所以它们是同义词，必然冲突。若关键字恰好都是 16 的倍数加常数，mod 16 会把它们全部挤在一起——这也是教材建议除留余数法的模数取“不太接近 2 的幂的质数”的原因。

3.10 第十章 国内外高校公开试题典型题专题

检索与核验说明：以下题目均来自可公开访问的大学课程页面或官方试卷档案，访问日期均为 2026 年 11 月 2 日；题目按原意翻译或概述（非整页复制）。国内高校的期末试卷官方公开较少，本章国内部分采用教育部考试中心命题的全国硕士研究生招生考试计算机学科专业基础（408）真题，其题面经多个公开来源交叉核对，来源可信度标为“可交叉核验的公开档案”。本次检索覆盖 5 个来源机构（印度 West Bengal State University、美国 University of Maryland、UC Berkeley、Princeton University、中国 408 全国统考），未达到 6 所，如实说明。

【高校公开试题 1】【公开真题】

学校：West Bengal State University（印度）｜课程：Computer Science CC5（CMSACOR05T），B.Sc. Honours 第 3 学期｜年份：2021–22 学年考试｜类型：期末考试｜题号：Q2(a)(b)(c)｜来源：`https://prtgc.ac.in/wbsu_previous_question_papers/computer-science/2021/CMSACOR05T%20Data%20structure%202021.pdf`（学院公开的历年试卷档案）｜对应知识点：BST 构造与比较次数｜对应原作业：8.1①｜难度：基础｜可信度：可交叉核验的公开档案

题意（翻译）：按 JAN, MAR, JUN, FEB, JUL, MAY, APR, SEP, AUG, OCT, NOV, DEC 的顺序建立 BST（5 分）；插入 NOV 需要多少次比较（1 分）；求建树过程中平均每个关键字所需的比较次数（2 分）。

为什么选：与 8.1① 是同一组数据、不同插入顺序——检验你是否真的理解“形状由插入顺序决定”。

解：逐个插入（大写字典序）：JAN 根；MAR 右；JUN（>JAN，<MAR）MAR 左；FEB JAN 左；JUL（<JUN）JUN 左；MAY MAR 右；APR FEB 左；SEP MAY 右；AUG APR 右；OCT SEP 左；NOV OCT 左；DEC AUG 右。所得树**与图 2 完全相同**（大写不改变比较结果）。插入 NOV 时依次与 JAN, MAR, MAY, SEP, OCT 比较，共 **5 次**。建树时第 i 个关键字的比较次数 = 其深度（根深度 0），总和

为内路径长度 $I = 30$，平均 $\boxed{30/12 = 2.5}$ 次（按“插入比较数”理解；若理解为建成后的查找成功 ASL，则为 3.5）。
与本次作业的联系：两个不同的插入序列生成了同一棵树，因为每个结点的祖先都比它先插入。
【易错点】“建树的平均比较次数”与“查找成功 ASL”差 1：前者在找到空位时不和自己比较。与原作业易错 5 同属“计数口径”问题。

【高校公开试题 2】【公开真题】

学校：同上（WBSU）｜课程：CMSACOR05T ｜年份：2021–22 ｜类型：期末考试｜题号：Q1(e) ｜来源：同上｜对应知识点：AVL 树高度上界｜对应原作业：8.1③、变式 1-3 ｜难度：基础｜可信度：可交叉核验的公开档案

题意：7 个结点的 AVL 树最大可能高度是多少?
解：由 $N_1 = 1, N_2 = 2, N_3 = 4, N_4 = 7, N_5 = 12$，$N_4 = 7 \le 7 < N_5$，最大高度为 **4**（按结点层数计；若按边数计则为 3）。
【易错点】高度定义（层数还是边数）因教材而异，答题时应先写明约定；不要用 $\lfloor \log_2 n \rfloor + 1 = 3$（那是完全二叉树的高度，即最小高度）。

【高校公开试题 3】【公开真题】

学校：University of Maryland, College Park（美国）｜课程：CMSC 420 Data Structures（Dave Mount）｜年份：Fall 2020 ｜类型：Midterm Exam ｜题号：Problem 2.3 ｜来源：`https://www.cs.umd.edu/class/fall2020/cmsc420-0201/Handouts/ex1.pdf`（课程主页）｜对应知识点：AVL 平衡条件｜对应原作业：8.1③｜难度：中等｜可信度：官方直接来源

题意（翻译）：在一棵合法 AVL 树中插入一个比所有关键字都小的关键字和一个比所有关键字都大的关键字，不做任何再平衡。结果一定仍是合法 AVL 树吗? 判断真假并简要说明。（关键词 justify：需要给出理由或反例。）
思路：“左右各加一个”看似对称，但两个新结点分别加在最左路径和最右路径的末端，可能让路径上某个内部结点失衡，而另一侧的插入根本不经过它。
解：**假**。反例：AVL 树 $3(2(1,-),4)$（各 BF：3 为 1，2 为 1）。插入 0 成为 1 的左孩子，此时结点 2 左高 2、右高 0，BF= 2；插入 5 成为 4 的右孩子，不影响结点 2。结果不是 AVL 树。
与本次作业的联系：与 8.1③ 插入 Nov 一样，失衡可能发生在路径中间的结点，而不只是根。
【易错点】只检查根的 BF（根为 $3 - 2 = 1$ 看似合法）。与原作业易错 2 同一机制。

【高校公开试题 4】【公开真题】

学校：University of Maryland ｜课程：CMSC 420 ｜年份：Fall 2020 ｜类型：Midterm Exam ｜题号：Problem 2.7 ｜来源：同上｜对应知识点：B 树关键字数下限｜对应原作业：8.2 ｜难度：基础｜可信度：官方直接来源

题意（翻译）：任意一棵含 n 个关键字（n 很大）的 5 阶 B 树中，某个结点可能存放的最少关键字数是多少？选项 1/2/3/4/5。
解：非根结点至少 $\lceil 5/2 \rceil = 3$ 棵子树，即至少 2 个关键字；但**根**只要求至少 2 棵子树，可以只有 1 个关键字。题目问“任何结点”，故答 **(a) 1**。
为什么选：它把 B 树定义中最容易忽略的“根的例外”单独拎出来考。8.2 删除 8（变式 2-2）后根恰好只剩 1 个关键字 [50]，正是此情形。
【易错点】答 2（忘了根的例外）。**快速检查**：问“结点”时先想想根。

【高校公开试题 5】【公开课程题】

学校：University of California, Berkeley（美国）｜课程：CS 61B Data Structures ｜年份：Spring 2021 ｜类型：Lecture 17 B-Trees 学习指南练习｜题号：Exercise 1 ｜来源：`https://berkeleycs-web-archive.vercel.app/cs61b/sp21/materials/lectures/lec17/lec17.html`（课程网站存档）｜对应知识点：2-3 树（3 阶 B 树）插入｜对应原作业：8.2①–③｜难度：基础｜可信度：官方镜像存档

题意（翻译）：画出依次插入 A, B, C, D, E, F, G 后得到的 2-3 树。

解：$[A, B]$；插 C 上溢 → B 上移：$B(A, C)$；插 D：$[C, D]$；插 E：$[C, D, E]$ 上溢 → D 上移，根 $[B, D]$；插 F：$[E, F]$；插 G：$[E, F, G]$ 上溢 → F 上移，根 $[B, D, F]$ 上溢 → D 上移成新根。结果为 $D\big(B(A, C),\ F(E, G)\big)$，一棵满二叉形状的 2-3 树（图 18）。

与本次作业的联系：与 8.2③ 一样出现“连锁分裂”，且这里根分裂使树长高。有序插入让普通 BST 退化（变式 1-2(a)），却让 B 树长得最整齐——这是理解“B 树只向上长”的好例子。

【易错点】根分裂时忘记新建根；或把上溢结点的第 1 或第 3 个关键字上移。

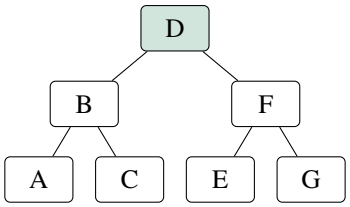


图 18: CS 61B 练习：依次插入 A–G 得到的 2-3 树。看点：最后一次插入引起两级分裂，根被“顶”出来，树高 +1。

【高校公开试题 6】【公开课程题】

学校：Princeton University（美国）｜课程：COS 226 Algorithms and Data Structures ｜年份：Spring 2009 ｜类型：Exercises on Hashing ｜题号：Exercise 2 ｜来源：`https://www.cs.princeton.edu/courses/archive/spring09/cos226/exercises/hash.html`｜对应知识点：线性探测构表｜对应原作业：8.3①｜难度：基础｜可信度：官方直接来源

题意（概述）：表长 $M = 16$，线性探测；把第 k 个字母映射为 $11k \bmod M$（A= 1, B= 2, …）；依次插入 R E P U B L I C A N S，给出最终表。

解：散列值：R(18)→6, E(5)→7, P(16)→0, U(21)→7, B(2)→6, L(12)→4, I(9)→3, C(3)→1, A(1)→11, N(14)→10, S(19)→1。冲突处理：U：7 被 E 占 → 8；B：6, 7, 8 被占 → 9；S：1 被 C 占 → 2。最终表：

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
键	P	C	S	I	L		R	E	U	B	N	A				

检验：程序模拟一致。**联系**：这里散列函数的值域与表长相同（均为 16），回绕不出现；对比 8.3 值域 16、表长 18 的设计，能更清楚看出“起点”与“地址空间”的区别。

【易错点】B 要连续探测 3 次（6, 7, 8 都已占用），容易只看到 6 被 R 占而放到 7。

【高校公开试题 7】【公开课程题】

学校：Princeton University ｜课程：COS 226 ｜年份：Fall 2024 ｜类型：Precept 6 讲义｜题号：Part B, Part 1 ｜来源：`https://www.cs.princeton.edu/courses/archive/fall24/cos226/precepts/precept6.pdf`｜对应知识点：线性探测 vs 链地址｜对应原作业：8.3、变式 3-2 ｜难度：基础｜可信度：官方直接来源

题意（概述）：容量 11，$h(i) = i \bmod 11$，依次插入 15, 90, 53, 100, 34, 65, 20。(1) 用线性探测时，哪些位置最终为空？(2) 用链地址法时各链表长度是多少？

解：散列值 15→4，90→2，53→9，100→1，34→1，65→10，20→9。(1) 34：1 占 → 2 占 → 3；20：9 占 → 10 占 → $(10+1) \bmod 11 = 0$。占用位置 0, 1, 2, 3, 4, 9, 10，**空位为 5, 6, 7, 8**。(2) 链长：位置 1 和 9 各 2 个，位置 2、4、10 各 1 个，其余为 0。
联系：同一组数据两种冲突处理并列，正是变式 3-2 的思路；20 的“绕回 0”与 8.3 中 63 的回绕同型。

【高校公开试题 8】【公开课程题】

学校：Princeton University ｜课程：COS 226 ｜年份：Spring 2016 ｜类型：Week 5 Activity（期中复习讲义）｜题号：Hashing 部分 (a)(c) ｜来源：`https://www.cs.princeton.edu/courses/archive/spring16/cos226/precepts/week5-handout.pdf` ｜对应知识点：线性探测性质与代价 ｜对应原作业：8.3④ ｜难度：中等 ｜可信度：官方直接来源

题意（翻译）：判断真假：(a) 若表长不小于关键字个数，线性探测总能为任何关键字找到位置；(c) 线性探测散列表半满时，查找命中平均需要 1.5 次探测。
解：(a) **真**：探测序列循环遍历全部地址，只要还有空位必能到达。(c) **真**（在 Knuth 近似公式意义下）：$\alpha = 0.5$ 时 $\frac{1}{2}\left(1 + \frac{1}{1-0.5}\right) = 1.5$。
联系：8.3 中 $\alpha \approx 0.61$，理论值约 1.79，实测 23/11≈2.09（图 15）。
【易错点】把“查找命中”的公式用于“查找未命中”（$\alpha = 0.5$ 时未命中约 2.5 次）。与原作业易错 4 的“成功/失败口径”同一机制。

【高校公开试题 9】【公开真题】（全国统考）

考试：全国硕士研究生招生考试计算机学科专业基础（408），教育部考试中心命题 ｜年份：2010 ｜题号：第 41 题（10 分）｜来源：题面经多个公开来源交叉核对（如 `https://ctyun.cn/zhishi/p-373731`）｜对应知识点：散列构表、装填因子、成功/失败 ASL ｜对应原作业：8.3、变式 3-1 ｜难度：中等 ｜可信度：可交叉核验的公开档案

题意：将关键字序列 $(7, 8, 30, 11, 18, 9, 14)$ 散列存储到下标从 0 开始的一维数组中，$H(key) = (key \times 3) \bmod 7$，线性探测再散列，装填因子 0.7。(1) 画出散列表；(2) 求等概率下查找成功和不成功的 ASL。
解：表长 $m = 7/0.7 = 10$。散列值：7→0，8→3，30→6，11→5，18→5，9→6，14→0。18：5, 6 占 → 7；9：6, 7 占 → 8；14：0 占 → 1。

地址	0	1	2	3	4	5	6	7	8	9
键	7	14		8		11	30	18	9	
比较次数	1	2		1		1	1	3	3	

$\text{ASL}_{\text{成功}} = \frac{1+2+1+1+1+3+3}{7} = \frac{12}{7}$。失败查找起点只能是 $0 \sim 6$（散列函数值域），计入与空单元的比较：起点 0→3 次，1→2，2→1，3→2，4→1，5→5，6→4，合计 18，$\text{ASL}_{\text{失败}} = \frac{18}{7}$（程序验证 ✓）。
为什么选：它把本次 8.3 的两个隐含点都显式化了——“表长由装填因子反推”“失败 ASL 的分母是散列函数值域大小 7 而不是表长 10”。
【易错点】失败 ASL 除以 10；或忘记把“遇到空单元”的那次比较计入（不同教材口径不同，408 标准答案计入）。与变式 3-1 完全同型。

跨校横向比较

知识点	来源	题型	主要方法	易错点	与原作业关系
BST	WBSU 2021	构造 + 计数	逐个插入、按深度求和	计数口径差 1	8.1 ①同数据
AVL	WBSU 2021; UMD F20	计算;判断 + 反例	N_h 递推；构造反例	只看根的 BF	8.1 ③
B 树	UMD F20; Berkeley S21	选择;作图	定义上下限；连锁分裂	根的例外；新根	8.2
散列	Princeton 09/16/24；408 2010	作图；判断；计算	线性探测模拟;近似公式	回绕、失败分母	8.3

观察：国外课程题更常以“判断真假并说明理由”“给出反例”考概念（UMD、Princeton），也会直接考近似公式的含义；国内 408 更偏完整计算与口径（装填因子反推表长、成功与失败 ASL 同时求）。两者共同的核心都是：**严格按定义模拟 + 明确计数口径**。

高频母题结构

母题	识别信号	标准方法	最易错	对应作业
按层求 ASL	等概率、查找成功、某种树	画树、按层计数	字典序；近似代精确	8.1 ①②
AVL 构造	依次插入、平衡二叉树	自底向上找第一个 ± 2	失衡点与型别	8.1 ③
B 树维护	m 阶、插入/删除序列	上溢分裂、下溢借/合	阶数含义、根的例外	8.2
开放定址散列	地址范围、$H(key)$、线性探测	逐个模拟探测	回绕模数、失败分母	8.3

章末：考试迁移总结

- **国内常见考法**（据本次检索的 408 真题）：给序列构造结构 → 画图 → 计算成功与失败 ASL；常与装填因子、表长反推结合。
- **国外常见考法**（据 UMD、Princeton、Berkeley 材料）：真假判断加理由、构造反例、给出插入序列画 2-3 树、考查性能公式含义。
- **共同核心**：定义要背准（阶、BF、装填因子），过程要模拟准，口径要写清。
- **复习优先级**：必须掌握——四个母题；必须会辨析——成功/失败 ASL、值域/地址空间、借/合并；必须会检查——中序有序、BF 合法、结点关键字数合法；值得挑战——UMD 2.3 的反例构造。

3.11 第十一章 生活中的实际应用

【生活应用 1】翻纸质词典——折半思想（类比理解）

场景：查一个英文单词时，翻开词典中间，看看在前半还是后半，再对半翻。**对应**：8.1② 折半查找。**机制**：每次比较排除约一半页码。**量化估算**：一本 1000 页的词典，按纯折半最多约 $\lceil \log_2 1001 \rceil = 10$ 次就能锁定页码（估算值）。**模型与现实的差距**：真人会根据首字母“估位置”（更像插值查找），而且不会严格对半；所以这是**类比**，不是说人脑在执行折半算法。**动手观察**：和同学玩“1–1000 猜数字”，只回答“大了/小了”，记录最多需要几次（图 19）。

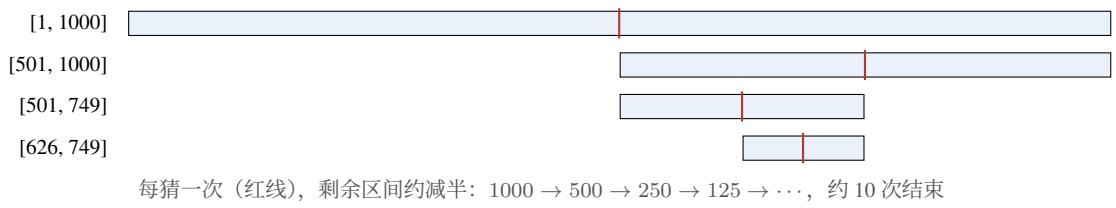


图 19: 猜数字游戏中区间的收缩。看点：区间长度按 1/2 的几何级数缩小，所以次数是对数级的。

【生活应用 2】教务系统按学号查成绩——数据库的 B⁺ 树索引（真实使用）

场景：在选课或成绩系统中输入学号，几乎瞬间返回结果。**对应**：8.2 B 树。**机制**：许多学校系统后台使用关系数据库；以 MySQL 为例，其官方文档说明 InnoDB 的索引（空间索引除外）采用 B-tree 数据结构，默认索引页大小为 16KB。一个页就是树的一个结点，查一条记录只需从根页向下读几页。**量化估算**：若每个内部页能放约 1000 个“键 + 指针”（估算值，取决于键长），三层就能索引约 $1000^3 = 10^9$ 条记录，即查一次最多读 3 个页。**模型与现实的差距**：实际使用的是 B⁺ 树（数据在叶子、叶子成链，见图 14），且上层页通常常驻内存；具体某所学校用什么数据库无法从外部确认，这里只说明主流数据库的通用做法（图 20）。

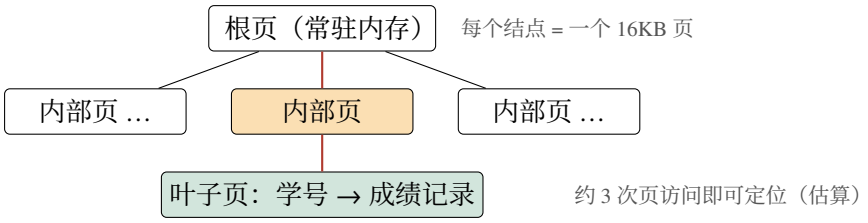


图 20: 数据库按学号查找时沿 B⁺ 树走过的页（红色路径）。看点：树“又胖又矮”，十亿级数据也只有三四层。

【生活应用 3】你写的程序里的“字典”——散列表（真实使用）

场景：用 Python 统计一篇文章每个单词出现几次，或用 C++ 的 `std::unordered_map` 做“学号 → 姓名”。**对应**：8.3 散列。**机制**：CPython 的 `dict` 源码（`Objects/dictobject.c`）采用开放定址法，探测序列是带扰动的伪随机序列而不是简单的 $+1$，目的正是减轻 8.3 中看到的聚集；C++ 标准规定的 `unordered_map` 接口以“桶”组织元素，主流实现采用链地址法。两者都在装填因子超过阈值时扩容并重新散列（对应图 15 中曲线陡升的部分）。**量化**：查找平均只需常数次探测，与 n 无关。**模型与现实的差距**：真实散列函数远比 *key* mod 16 复杂，且会动态扩容（图 21）。

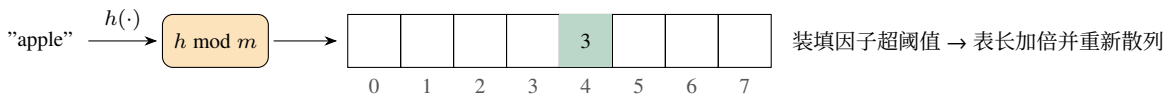


图 21: 程序中字典的工作方式：先把关键字变成整数，再取模定位。看点：与 8.3 完全同构，只是散列函数更好、并会自动扩容。

【生活应用 4】图书馆找书——多路分层索引（类比理解）

场景：按索书号找书：先看楼层索引牌，再找书架区，再找书架层，最后在一层里顺序找。**对应**：8.2 B 树的“多路分层”思想。**机制**：每一层索引把范围切成很多段（多路），而不是只切两半，所以层数很少——这正是 B 树“胖而矮”的直觉。**量化估算**：若 5 层楼 × 每层 40 个书架 × 每架 6 层，三级索引即可把范围缩小到一层书架（估算值）。**差距**：书架不会自动“分裂与合并”，所以这只是**类比**，不是 B 树算法本身（图 22）。

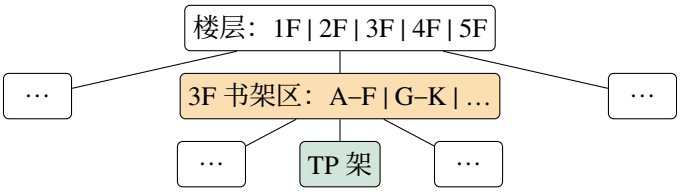


图 22: 图书馆的分层索引。看点：每一级都是“多路分叉”，与 B 树结点“一次比较多个关键字、走向多个孩子”相似。

知识点	生活场景	起作用的方法	类型	对应原题
折半查找	翻词典、猜数字	每次排除一半	类比	8.1 ②
B/B$^+$ 树	教务系统按学号查成绩	页式多路平衡树索引	真实使用	8.2
散列表	Python `dict`、C++ `unordered_map`	散列定位 + 冲突处理 + 扩容	真实使用	8.3
多路分层	图书馆找书	分级索引	类比	8.2

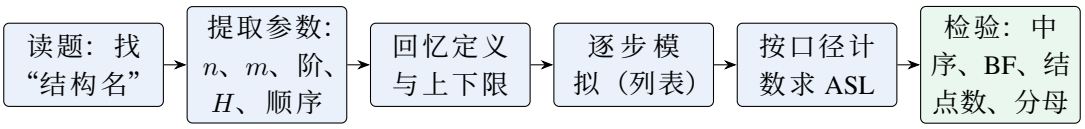
3.12 第十二章 学习总结

必须掌握：(1) ASL $=\frac{1}{n}\sum$(层次)；(2) BST 插入规则与字典序；(3) 折半判定树按层计数；(4) AVL：BF、最小不平衡子树、四型旋转；(5) m 阶 B 树的关键字数上下限、分裂、借、合并；(6) 线性探测的探测序列、成功与失败 ASL。

建议重点复习：8.1③ 插入 Nov 的全过程；8.2③ 的连锁分裂与 ④ 的借位；8.3 中 63 的回绕；变式 2-1、3-1。

必须避免的错误：按语义而非字典序比较字符串；在 BF$=\pm1$ 的结点旋转；把“3 阶”理解为 3 个关键字；兄弟有余却合并；回绕时模 16；失败 ASL 除以关键字数。

做题识别流程：



思维链：看到“依次插入 + 平衡二叉树”$\rightarrow$ 想到 BF 与四型旋转 $\rightarrow$ 每步自底向上找第一个 ±2 $\rightarrow$ 检查中序与 BF；看到“m 阶 B 树”$\rightarrow$ 写出关键字数范围 $[\lceil m/2\rceil-1,\ m-1]$ $\rightarrow$ 上溢分裂、下溢先借后合 $\rightarrow$ 检查等高与有序；看到“地址范围 + 哈希函数 + 线性探测”$\rightarrow$ 分清值域与地址空间 $\rightarrow$ 模表长探测 $\rightarrow$ 成功除以 n、失败除以值域大小。

4 第四部分　参考资料与权威来源

网页资料访问日期均为 2026 年 11 月 2 日。

教材与经典著作

[1] 严蔚敏，吴伟民.《数据结构（C 语言版）》，清华大学出版社. 第 9 章“查找”：ASL 定义、二叉排序树、平衡二叉树（四种旋转）、B 树定义及插入删除、散列表与线性探测再散列。（用于统一定义与符号约定）

[2] D. E. Knuth. *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998. §6.2.1（折半查找及其历史）、§6.2.3（平衡树）、§6.4（散列、线性探测分析与历史注记）。历史注记节选页：https://www.gwern.net/doc/cs/algorithm/information/compression/1998-knuth-taocp-v3-sortingandsearching-hashinghistory.pdf

[3] R. Sedgewick, K. Wayne. *Algorithms*, 4th ed., Addison-Wesley, 2011. §3.3（2-3 树与平衡查找树）、§3.4（散列表）。（用于第 8.2 题“2-3 树视角”）

原始论文与史料

[4] G. M. Adel’son-Vel’skii, E. M. Landis. An algorithm for organization of information. *Doklady Akademii Nauk SSSR*, 146(2):263–266, 1962. 题录：https://m.math-net.ru/eng/dan26964

[5] R. Bayer, E. McCreight. Organization and Maintenance of Large Ordered Indices. Boeing Scientific Research Laboratories, Math. and Info. Sciences Report No. 20, July 1970；正式发表于 *Acta Informatica* 1:173–189, 1972. 报告扫描件：https://infolab.usc.edu/csci585/Spring2010/den_ar/indexing.pdf

[6] H. Stevens. Hans Peter Luhn and the Birth of the Hashing Algorithm. *IEEE Spectrum* 55(2):44–49, 2018. https://spectrum.ieee.org/hans-peter-luhn-and-the-birth-of-the-hashing-algorithm

[7] NIST Dictionary of Algorithms and Data Structures：AVL tree 与 hash table 条目（含原始文献信息）。https://xlinux.nist.gov/dads/HTML/hashtab.html

[8] Wikipedia：B-tree、AVL tree、Binary search tree 条目（用于交叉核对发明年代与人物，人物与年代均已与上述原始文献/Knuth 记述对照）。https://en.wikipedia.org/wiki/B-tree, https://en.wikipedia.org/wiki/AVL_tree, https://en.wikipedia.org/wiki/Binary_search_tree

[9] IT History Society：Georgy Adelson-Velsky 条目（Kaissa 象棋程序）。https://ithistory.org/node/30541

[10] 折半查找历史（Mauchly 1946、Lehmer 1960）的二次核对：arXiv:1506.08620 引言部分对 Knuth 记述的转述。https://arxiv.org/abs/1506.08620

高校公开试题与课程资料（第十章）

[11] West Bengal State University, B.Sc. (Hons.) Computer Science, 3rd Semester Examination 2021–22, CMSACOR05T (CC5) Data Structure，Q1(e)、Q2(a)–(c)。https://prtgc.ac.in/wbsu_previous_question_papers/computer-science/2021/CMSACOR05T%20Data%20structure%202021.pdf

[12] University of Maryland, CMSC 420 (Fall 2020, Dave Mount), Midterm Exam, Problems 2.3、2.7。https://www.cs.umd.edu/class/fall2020/cmsc420-0201/Handouts/ex1.pdf

[13] UC Berkeley, CS 61B (Spring 2021), Lecture 17 B-Trees Study Guide（课程网站存档）。https://berkeleycs-web-archive.vercel.app/cs61b/sp21/materials/lectures/lec17/lec17.html

[14] Princeton University, COS 226 (Spring 2009), Exercises on Hashing。https://www.cs.princeton.edu/courses/archive/spring09/cos226/exercises/hash.html

[15] Princeton University, COS 226 (Fall 2024), Precept 6。https://www.cs.princeton.edu/courses/archive/fall24/cos226/precepts/precept6.pdf

[16] Princeton University, COS 226 (Spring 2016), Week 5 Activity。https://www.cs.princeton.edu/courses/archive/spring16/cos226/precepts/week5-handout.pdf

[17] 全国硕士研究生招生考试计算机学科专业基础（408）2010 年第 41 题，题面交叉核对来源：https://ctyun.cn/zhishi/p-373731；https://damodev.csdn.net/6a437c6610ee7a33f284dda6.html（解答为本讲义独立推导并经程序验证，未采用该页的失败 ASL 表格）

[18] MIT OpenCourseWare, 6.006 Introduction to Algorithms (Spring 2020), Lecture 7: Binary Trees, Part 2: AVL（用于核对 AVL 旋转与高度论证）。https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_lec7/

生活应用相关的官方文档

[19] MySQL 8.0 Reference Manual, *The Physical Structure of an InnoDB Index*（InnoDB 索引为 B-tree 结构；默认索引页 16KB）。https://dev.mysql.com/doc/refman/8.0/en/innodb-physical-structure.html

[20] CPython 源码 `Objects/dictobject.c` 中关于开放定址与扰动探测的注释。https://github.com/python/cpython/blob/main/Objects/dictobject.c

计算核验说明

第二部分全部答案（BST 层次与 ASL、折半判定树、AVL 旋转序列与 ASL、3 阶 B 树四步结果、散列表与 ASL）以及第三部分全部变式和第十章各题的数值，均由本讲义作者用 Python 独立编写的模拟程序（BST/AVL 插入、3 阶 B 树插入与删除、线性探测与链地址散列）逐一复算，结果与正文一致。